Cluster Computing (2025) 28:67
https://doi.org/10.1007/s10586-024-04712-z

An efficient deep reinforcement learning based task scheduler in cloud-fog environment

Prashanth Choppara[1] · Sudheer Mangalampalli[2]

Received: 22 July 2024 / Revised: 27 August 2024 / Accepted: 14 September 2024 / Published online: 5 November 2024

Abstract
Efficient task scheduling in cloud and fog computing environments remains a significant challenge due to the diverse nature and critical processing requirements of tasks originating from heterogeneous devices. Traditional scheduling methods often struggle with high latency and inadequate processing times, especially in applications demanding strict computational efficiency. To address these challenges, this paper proposes an advanced fog-cloud integration approach utilizing a deep reinforcement learning-based task scheduler, DRLMOTS (Deep Reinforcement Learning based Multi Objective Task Scheduler in Cloud Fog Environment). This novel scheduler intelligently evaluates task characteristics, such as length and processing capacity, to dynamically allocate computation to either fog nodes or cloud resources. The methodology leverages a Deep Q-Learning Network model and includes extensive simulations using both randomized workloads and real-world Google Jobs Workloads. Comparative analysis demonstrates that DRLMOTS significantly outperforms existing baseline algorithms such as CNN, LSTM, and GGCN, achieving a substantial reduction in makespan by up to 26.80%, 18.84, and 13.83% and decreasing energy consumption by up to 39.60%, 30.29%, and 27.11%. Additionally, the proposed scheduler enhances fault tolerance, showcasing improvements of up to 221.89%, 17.05%, and 11.05% over conventional methods. These results validate the efficiency and robustness of DRLMOTS in optimizing task scheduling in fog-cloud environments.

Keywords Fog computing · Reinforcement learning · Makespan · Energy consumption · Fault tolerance

1 Introduction

Over the course of the year, a rising number of smart devices become internet enabling, which enables more sophisticated communication and coordination between them and their users. Smart devices (such as tablets, mobile phones, motors, sensors, relays and actuators) connected through the Internet of Things (IoT) are estimated to reach 75 billion by 2025. They produced an immense measure of information from 1.1 Zettabytes (or 88 Exabyte's) each year in 2021 to around 3.2 Zettabytes (or 195 Exabyte's) each year by 2025 [1]. Managing such a lot of information was the greatest test with the conventional IoT-and Cloud-based frameworks. Task scheduling sign up the cloud offers numerous benefits and has some disadvantages like latency and network dependency, unpredictable performance, complexity, data security and privacy. To overcome this issue, a decentralized computing paradigm that as in goal to bring computation and data storage closer to data resource, reducing the latency and addressing disadvantages associated with cloud computing [2]. Fog computing emerged as an alternative to cloud computing. It was a response to the consumers' desire for complex applications, using a new computer paradigm. Fog computing facilitates the networking, data processing, analysis of data. The objective of cloud-like services is to bring IoT devices closer in order to minimize execution time, save energy, optimize bandwidth use, and enhance privacy for

✉ Prashanth Choppara
choppara.22phd7109@vitap.ac.in

✉ Sudheer Mangalampalli
ms.sudheer@manipal.edu

[1] VIT-AP University, Amaravati, Andhra Pradesh 522237, India

[2] Department of Computer Science & Engineering, Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education, Manipal, India

IoT applications [3]. The effectiveness of the fog-computing paradigm primarily depends upon task scheduling, how scheduling managing the tasks and scheduling them on to proper virtual resources. It also affects the different parameters i.e. makespan and fault tolerance this will impact the performance, reliability and security of cloud, fog and user. An effective scheduler assigns the tasks in a virtual machines effective manner its leads to reduce the parameters like makespan, energy consumption, fault tolerance [4]. Energy consumption is critical aspect in fog computing, determining its efficiency, sustainability, and cost-effectiveness. Fog computing aims to boost the resource utilization and monitoring the electrical power needed by a network of interconnected devices such as sensors, edge servers, and networking equipment [5]. This conscientious approach not only matches with environmental sustainability goals, but it also leads to longer battery life for edge devices, which is critical in scenarios that rely on battery-powered sensors. Furthermore, reducing energy consumption results in significant cost savings, particularly in large-scale deployments where operational charges might be significant. Furthermore, optimized energy consumption leads to increased system dependability and redundancy, which is critical in dispersed fog computing systems [6]. Failures and errors happened, which may need a recovery process and more time to complete the task. When nodes can join or leave the network, delaying jobs while they are reallocated to other nodes due to failures can have an influence on metrics like redundancy, which help the system recover rapidly from node failures. It will occur in this scenario as a result of incorrect job scheduling for virtual machines. Many authors used nature inspired and heuristic techniques [7–13] for tackle the task scheduler but still there is a lack of an effective scheduler while minimizing the metrics makespan, energy and fault tolerance. Machine learning plays a suggestive role in task scheduling in fog and cloud computing. In this process most data was coming from IoT devices, which is connected, to cloud [14, 15]. In this paper, we are formulated the ML based Deep Reinforcement Learning (DRL) technique to schedule the tasks in fog layers to minimize makespan, fault tolerance, consumption of energy. Therefore, we have used the DRLMOTS scheduler, which fed priorities to task manager and generates schedules by considering priorities while minimizing the metrics like makespan, fault tolerance, energy consumption.

The main contributions of this paper as follows:

1. Task scheduling algorithm DRLMOTS proposed to generate schedules either in fog/cloud dynamically according to the priorities fed to task manager.
2. DQN model is integrated into scheduler to generate decisions based on the priorities fed to scheduler.
3. Extensive Simulations conducted on simpy using workloads taken from google jobs dataset and carried out simulation by varying and fixed number of VMs for evaluating parameters i.e. makespan, fault tolerance, energy consumption.
4. To check the efficacy of proposed scheduler model DRLMOTS, it was compared over existing the state of art approaches i.e. CNN, LSTM, GGCN algorithms evaluate above-mentioned parameters.

Rest of the manuscript is arranged as follows: Existing research is presented and correlated in Sect. 2, Fog system architecture and task scheduling problem formulation in Sects. 3 and 4, working of DRLMOTS and methodology used in DRLMOTS discussed in Sect. 5, Experimental results discussed in Sect. 6, Conclusion and Future work discussed in Sect. 7.

2 Related work

This section reviewed various works done by other researchers on cloud—fog computing fields. Iftikhar et al. [7] formulated Hunter plus Artificial intelligence-based energy effective errand booking for cloud—haze processing conditions which inspects the bidirectional gated intermittent unit to advance the task scheduling for scheduling simulation work carried out the pyfog to identified the job completion [7]. Movahedi and Defude [8] formulated the two approaches one is integer (LPO) linear programming optimization consider time, fog layer energy consumption, second approach called opposition based (CWOA) chaotic whale optimization algorithm to solve the cloud paradigm. The simulation work carried out the python language and proposed algorithm was compared to ABC, PSO, GA [8]. Researcher [9] designed the simulation in ifogsim toolkit he proposed the task-scheduling algorithm moth-flame optimization (MFO) to minimizing execution time and energy consumption. Proposed algorithm was compared to existing algorithms first come first serve, partially swam optimization [9]. Yadav et al. [10] formulated the hybrid approach using heuristic and metaheuristic for task scheduling hybrid approach bi objective optimization approach was proposed to minimize metric makespan, cost, throughput. The results are compared with the GA, PSO, HEFT. It is simulated java programming [10].

Author [11, 12] formulated Task scheduler which used stochastic learning compared to the existing baseline algorithms achieve the reduce the fault tolerance, response time, energy computation of proposed algorithm. It was

simulated using python programming. Author formulated the artificial ecosystem-based optimization and SSO to AEOSS scheduler better performance of metrics are makespan and throughput compared the experimental results are AEO, PSO, HHO, SSA, FA. MATLAB R2018b simulates. Montazerolghaem et al. [13] formulated the resource task-scheduling algorithm Harris Hawks's optimizer reduce the energy consumption, makespan and increased reliability. The proposed algorithm was compared existing algorithms WOA, FA, PSO shows the better results. The simulation work carried out the MATLAB 2018b [13].

Swarup et al. [14] simulated the experimental results using clouds. Author planned the q-learning based task booking application to limit the computational expense and energy utilization. The outcomes thought about the current calculations, for example, space shared booking, time-shared planning, and arbitrary planning. Q picking up planning given the better performance in service level agreement and cost, energy consumption [14]. Razaq et al. [15] formulated the reinforcement learning based technique like fragmented q—learning which is developed in python. Existing algorithms compared proposed approach given better performance of load-balancing and average delay [15]. In Cheng et al. [16] proposed a planning calculation that oversees QoS, VM cost and reaction time for the booking model. The structure utilizes a DQN that works on support learning. The whole trial was completed on a simpy, and it was looked at against irregular, cooperative effort (RR), and earliest schedulers, that DQN outperforms the listed algorithms for the specified parameters [16]. In Tong et al. [17] HEFT and Q-Learning algorithms. This procedure was carried out in two steps. Q Learning will sort tasks in the first phase to ensure effective work allocation. HEFT was used to allocate processors in the second phase. Cloudsim was used to simulate results. It compared to existing HEFT and CPOP algorithms. Finally, this strategy was proven to outperform prior approaches in terms of makespan.

In Choppara and Mangalampalli [18] formulated the reliability and trust aware task scheduler for cloud fog computing using advantage actor critic (A2C) algorithm it presents an innovative methodology RTATSA2C create to improve the performance and sustainability of cloud fog environment. Imposing the concepts through a reinforcement learning algorithm called advantage actor critic (A2C) that dynamically distributes the tasks to the cloud fog nodes based on their computational demands and the status of the network. The goal of RTATSA2C is to maximize important system parameters, including makespan, reliability, scalability and trust which are necessary for managing diverse unpredictable and time-sensitive tasks. The proposed scheduler does this by dynamically changing the number of VMs and integrating actor critic policies to carry out tasks and optimally utilize resources. Through the simulation analysis using Simpy framework and actual google cloud jobs data it can be seen that the RTATSA2C solution will have evidently less task completion time and far between system availability and fault tolerance than current algorithms such as LSTM, RIWOA, and DQN. Because of this RTATSA2C is well suited as a solution to improve the task scheduling in complex environments of cloud fog computing.

In Gazori et al. [19] formulated lowering service delays and processing costs, an effective scheduling system in a fog environment was designed. When tested against the FF, GS, RS algorithms, Deep Q-learning (DQL), double Q-learning procedures were implemented on Ifogsim and showed a considerable improvement over current technique. [19]. In Sharma and Garg [20] created an energy-efficient job scheduler based on RANN model. The dataset contains 18 million instances. Task scheduler outperforms existing models in terms of time utilization, required active racks, and execution overhead after being simulated on MATLAB and assessed against existing techniques [20]. In Siddesha et al. [21] an energy-conscious task-scheduling model that created to minimize energy consumption, makespan, resource utilization via DRL approach [21]. It was compared to SOTA techniques, and the deep reinforcement strategy outperformed for the above parameters.

In [22, 23] author formed to decrease energy use, expenses, and administration delays; a planning strategy called Cut Twofold Profound Q-learning was created. It utilizes target organizations and experience replay procedures. It is simulation framework built on the SimPy Python library. By minimizing the cost, service latency, and energy usage, Double Deep Q-learning outperforms compared approaches when compared to FCFS, random scheduling, and simulation results. The author developed an innovative and efficient reinforcement-learning method that integrates optimization techniques with reliable lower limits. Utilize reinforcement learning (RL) to determine process of selecting tasks. Dataset has 600 jobs that arrive during a time span of 1000 s, and it replicates the functionality of the SimPy framework in Python. Results are compared to baseline method MIN-MIN, which is a stochastic strategy aimed at minimizing task execution time. In [24] authors created a scheduling method that takes account the load-balancing and average delay under various IOT job counts. The job scheduling issue is addressed using the Markov decision process (MDP). The workloads used for this approach and all simulations run using the Python package fogpy are randomized worklogs. Simulation findings demonstrated that the authors' proposed technique outperformed existing parameters average

end-to-end delay for both IoT task splitting and load balancing considerations.

In [25] a task-scheduling method was designed by the author with the aim of balancing carbon emission ratio and makespan on pareto optimality. The polynomial mutation mechanism (PMM) was used to simulate the algorithms of marine predators. The results demonstrated that the marine predator algorithm's polynomial mutation mechanism had a significant influence by modifying the energy, makespan characteristics. In Mangalampalli et al. [26] review conducted the improper assignment of tasks to virtual resources VMs results in a reduction in service quality, violation of the SLA parameters, and ultimately a decrease in the cloud user's trust in cloud provider. Thus, we provide an effective task scheduling method that takes into account both the priority of individual tasks and virtual machines in order to effectively schedule jobs to the proper VMs and maintain trust in the cloud provider. In Mangalampalli et al. [27] formulated the cloud model; scheduling is a huge difficulty since it requires an effective scheduling mechanism that dynamically allocates resources to users based on their related demands in order to map heterogeneous tasks that originate from multiple sources. Insufficient planning inflates costs, utilizes more energy, and disregards the SLA settled upon between the cloud client and specialist co-op, bringing down assistance quality and client trust in the cloud specialist organization.

Table 1 represents summary of various scheduling algorithms focused on parameters makespan, cost and other operational costs but typically in cloud-fog environment assignment of tasks either to cloud or fog nodes is still challenging because of resource, computing constraints which may lead to increase in failure rate. Therefore, in this research, we carefully identified priorities of tasks based on their runtime capacities and fed to DRLMOTS scheduler which uses DQN model to accommodate tasks effectively to cloud/fog nodes considering above said constraints while addressing makespan, Fault tolerance, energy consumption.

3 Fog system architecture

The fog computing is not a replacement of cloud computing but it as adjunct technology that enhances the functionality of cloud services and takes them to the edge of an organization network. Fog computing refers to data processing, storage and analytics carried out near the originating devices hence reducing delays, bandwidth usage. This kind of approach is more advantageous when it comes to real rime applications like self-driving cars, robotics manufacturing and smart cities, among others since time is very essential here. Thus, cloud computing is a distributed, efficient and adaptable model for string huge files as well as for large computations but hear the latency bandwidth issues arise when the data are to be processed in central servers. To overcome these challenges solutions such as fog computing are employed in which preliminary data processing and decision making are carried out at local or at the edge of the data source. But it does improve real-time as well as decreasing the number of messages that need to be sent to the cloud for further processing and wasting bandwidth and cost. For example, in smart city environment the fog nodes work on certain real time operations which would include traffic monitoring and automatic control of the street lights. The other hand cloud computing can handle more complicated operations that require aggregation such as aggregating data over time to predict means of city planning or to undertake predictive maintenance analytics over used roads. Fog computing can then be integrated with cloud computing where it applies medium to large scale workloads that required real time responses than the cloud computing approach.

The proposed fog-cloud computing system is structured to efficiently manage task scheduling by integrating different components: IoT devices, fog nodes, cloud nodes. The architecture is divided into three primary layers:

IoT Device Layer: It comprises various IoT devices such as sensors, cameras,other smart devices that generate tasks. These tasks vary in computational needs and sensitivity to latency, depending on the specific device and application.

Fog Layer: Situated close to the IoT devices, fog nodes provide real-time processing capabilities. While they have limited computational power and storage, their low latency makes them ideal for tasks requiring immediate attention. Examples include edge servers and local data centers.

Cloud Layer: Located centrally, cloud nodes offer significant computational power, storage capacity, making them suitable for handling complex, resource-intensive tasks. However, they operate with higher latency compared to fog nodes. Typical examples of cloud nodes include large-scale data centers and cloud computing platforms.

The fog system architecture 3-tier in each phase some operations exist Fig. 1 phase one consists of departments and access points. Each department generates tasks based on user requirements. Every three departments transfer the data to access points. The access points combine the three departments' data. The second phase gets the tasks from access point to end node have some operation of tasks. This data was transferred to the fog nodes the resource allocation to VMs using the DRLMOTS scheduler $FogNode1 = \{vm_1, vm_2, vm_3, vm_4, \ldots\ldots\ldots\ldots\ldots vm_n\}$

Table 1 Represented the various task scheduling approaches using Machine Learning (ml) techniques

Authors	Techniques	limitations	Simulation tools	Parameters
[7]	Bi directional gated recurrent unit	High complexity in training, Limited to specific use cases	PyFog	Energy consumption, Job completion
[8]	Linear programing, opposition-based chaotic whale optimization	Not scale well with very large datasets, Susceptible to local optima	python	Energy consumption and Execution Time
[9]	Moth-flame optimization	Prone to slow convergence, Not handle dynamic changes well	IFogsim toolkit	Energy Consumption and Execution time
[10]	Bi-objective optimization	Computationally expensive, Not balance all objectives effectively	Java programming	Makespan, cost, Throughput
[11]	Stochastic learning	High computational overhead, May not be robust to all types of faults	python	Fault tolerance, Energy consumption response time
[12]	AIOSSA	Requires fine tuning of parameters, can be computationally intensive	MATLAB R2018b	Makespan, Throughput
[13]	Harris Hawks optimizer (HHO)	Performance depends on parameters settings Not perform well in all scenarios	MATLAB R2018b	Makespan time, Energy Consumption
[14]	Q-learning scheduling	High exploration time, struggle with very large state spaces	Python	cost and energy Consumption
[15]	Fragmented Q-learning	Not Scale well, High computational complexity	simpy	Load-balanced and average delay
[16]	DQN	Requires large amount of data, computationally intensive	Python	Success rate, QoS, response time
[17]	QL-HEFT	May not adapt well to dynamic changes, High computational overhead	cloudsim	Energy consumption, cpu utilization
[18]	RTATSA2C	Complexity in dynamically allocating tasks between cloud and fog nodes requires accurate real time state representation and feedback for optimal decision making	Simpy	Makespan, Reliability, Scalability, Trust, Fault tolerance
[19]	Deep Q-learning double queue learning	High computational requirements, Not handle real time changes well	ifogsim	Delay of services and computational Cost
[20]	RANN	Complex to implement, require significant computational resources	MATLAB	Makespan, energy consumption, Execution overhead
[21]	Deep reinforcement approach	Requires extensive training data, computationally intensive	MATLAB	Makespan, throughput, response Utilization, Energy consumption
[22]	Clipped Double Deep Q-learning using target network(CDDQL)	Not perform well in highly dynamic environments, High computational complexity	Simpy	Service delay, energy consumption
[23]	Reinforcement learning algorithm	High computational cost, Not scale well	Simpy	Task runtime
[24]	Markov decision process	Not handle large state spaces well, computationally intensive	Fogpy	Load balancing, average end-to-end delay
[25]	Marine predator algorithm with the polynomial and mutation mechanism (MPAPMM)	Sensitive to parameters settings, Not always find global optima	Jmetal framework	Makespan, energy Consumption

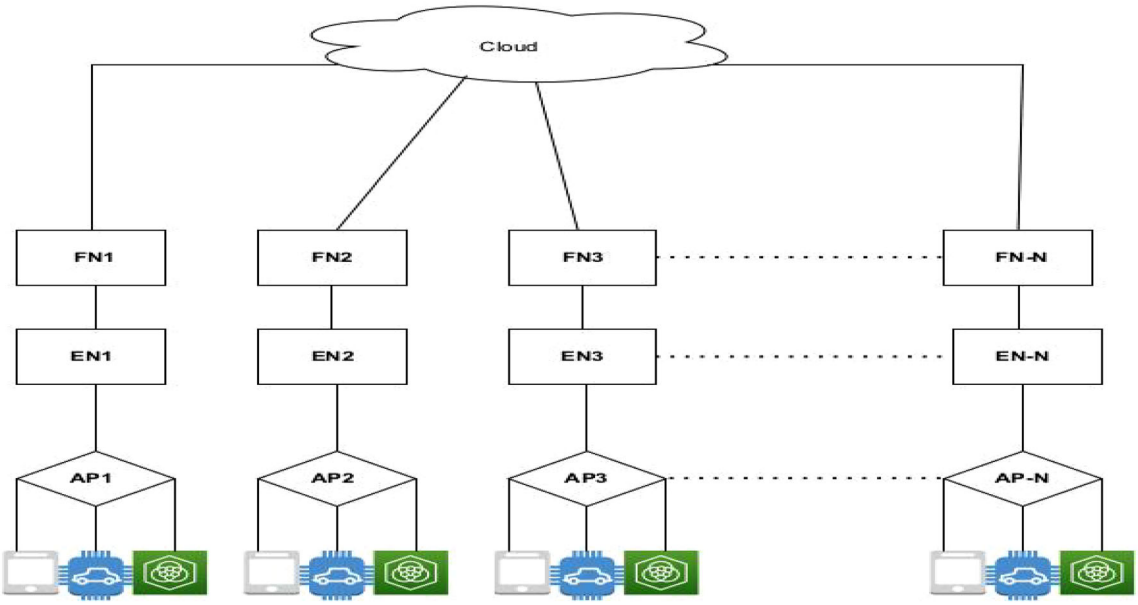

Fig. 1 Fog system architecture

Where vm_n is total number of VMs, $VM_{resources} = CPU_i, Storage_i, Memory_i$. Priorities of incoming tasks are examined and supplied into the DRLMOTS model, which determines scheduling decisions based upon the future and present running the tasks in underlying resources, minimizing makespan, fault tolerance, energy consumption (see Table 2).

4 Task scheduling system architecture

In the proposed fog-cloud computing framework, task scheduling was a crucial issue that must account for the heterogeneity of resources. The primary objective is to optimize allocation of tasks generated by IoT devices to either fog or cloud nodes, based on their computational demands and latency requirements. Several key factors are considered in formulating this problem. First, resource availability is critical in fog nodes have limited computational power and storage, whereas cloud nodes have abundant resources. Continuous monitoring of these resources is necessary to make informed scheduling decisions. Second, latency sensitivity varies among tasks. Real-time tasks, such as video streaming and online gaming, require immediate processing and low latency, making fog nodes the ideal choice. In contrast, tasks like data analysis and batch processing can tolerate higher latency and are thus better suited for cloud nodes. Third, energy consumption must be minimized to maintain system sustainability. Fog nodes generally consume less energy due to their proximity to IoT devices, while cloud nodes, despite being more power-intensive, provide higher computational

Table 2 Notation's used in proposed System Architecture

Notation's	Meaning
r	Random
d_r	Devices
t_r	Tasks
env	environments
AP_n	Access points
EN_n	End nodes
VM_n	Virtual machine
FN_n	Fog nodes
lo^{vmq}	Load on VM queue
t_h	Threshold value
H_p	Physical host
t_{ij}	Execution time
t_{ik}	Execution time
$\exists f \epsilon F$	VM queue is full
m	Makespan
F_i	Fault tolerance
E	Energy
a_t	Action of tasks
s_t	Current state of tasks
ret_t	Rewards of tasks
$s_t^{'}$	Next state
γ	Discount factor
α	Learning rate
ϵ	Exploration rate
ϵ_{decay}	Decay rate for exploration
ω	Replay buffer

efficiency for demanding tasks. Finally, network conditions, including bandwidth and congestion, affect scheduling decisions. Tasks should be distributed in a manner that minimizes network congestion and ensures efficient data transfer. So we formulated a cloud-fog framework to minimized the objectives.

IoT device generate the tasks through gateway the tasks are connect to the access point Fig. 1. This access point collects the three devices' tasks and calculates each task processing time using the task length. The End node collects these requests and prioritizes all jobs based on task length and processing capacity. Finally, all tasks are sent to the fog layers to schedule the tasks. It will then be input into the proposed model DRLMOTS, which is connected to the scheduling model, depending on job priorities. Tasks must be scheduled appropriately onto the VMs based on the suggestions of the scheduler, which is connected with the proposed model. Initially, the scheduler must transmit these prioritized jobs to executed the queue and then to the VMs. In this proposed architecture, our scheduler must keep tracking of coming requests and inform the resource manager about virtual resources. Model maintains the record of tasks coming requests, executed the requests in virtual machines, and virtual resources in resource management for each time interval of scheduler, which comprises the proposed model. As a result, based on these criteria, the scheduler will make a dynamic choice, such as mapped a task to a new VMs, mapped a job to existing VMs, or transferring existing tasks to other VMs, if a VM has enough storage, processing capacity to accept ongoing tasks. We employed the DQN as an approach based on rewards function tasks where these tasks are assigned to VMs. It will update the scheduling decisions to the scheduler and will take care of scheduling tasks appropriately in VMs. The primary goal of this scheduler is to properly map tasks to VMs based on their priority while minimized the metrics such as makespan, energy consumption, fault tolerance.

In Cloud-fog computing, efficient management and processing of tasks are critical for optimizing performance and resource utilization. The following set of equations models the generation, assignment, and processing of tasks within an IoT-fog computing environment. Each equation is explained in detail to provide a clear understating of its role and significance

$$d = \sum_{r=1}^{n} d_r \tag{1}$$

This Eq. (1) aggregates total number of IoT devices across various categories. Here, d represents the total number of devices d_r is the number of devices in the r-th category, and n denotes total number of categories. This aggregation is fundamental for understanding the scale of devices generating tasks in the network.

$$t = \sum_{r=1}^{n} t_r \tag{2}$$

This Eq. (2) presents total number of tasks generated by IoT devices. In this context, t indicates total number of tasks, while t_r represents tasks generated by devices in the r-th category. The parameter n represents number of categories. This step is essential for quantifying the workload generated by the IoT devices.

$$env = \sum_{r=1}^{n} d_r + t_r \tag{3}$$

The simulation environment, represented by env, combines both number of devices and number of tasks. It integrates d_r, the number of devices in the r-th category, and t_r, the number of tasks in the r-th category. This comprehensive view is crucial for simulating the operational environment accurately.

$$\sum_{r=1}^{n} AP_n \tag{4}$$

$$\sum_{r=1}^{n} AP_n = \sum_{i=1}^{n} d_n + \sum_{i=1}^{n} (t_{1i} + t_{2i} + t_{3i} \ldots\ldots\ldots\ldots\ldots + t_{ri}) \tag{5}$$

These equations detail how tasks and devices are aggregated at access points. The access point AP_n collects the total number of devices d_n and the tasks from different categories $(t_{1i} + t_{2i} + t_{3i} \ldots\ldots\ldots\ldots\ldots + t_{ri})$. This aggregation is vital for organizing the tasks before they are processed further

$$length = \sum_{i=1}^{n} t_n \tag{6}$$

This equation calculates the total length of tasks assigned to the access points. The term length represents the cumulative task length, while t_n denotes number of tasks in the n-th category. Assigning a length to each task is necessary for determining subsequent processing times.

$$EN = \sum_{i=1}^{n} EN_n \tag{7}$$

This equation indicates the total process time for tasks at the end nodes. Here, EN refers to the end nodes, and EN_n denotes the process time for tasks at the n-th end node. Calculating the process time is essential for effective task scheduling and resource allocation. The value selected based on empirical observations of our system average processing speed where it was determined that processing

1000 bytes of data typically requires 2 ms. By fixing this value the model achieves standardization and simplification ensuring consistent calculations across different scenarios. Additionally, we use this fixed value aligns with established benchmarks in fog computing research, providing a reliable and reproducible method for estimating processing time based on task length.

$$\sum_{i=1}^{n} EN_n = length \times \frac{2}{1000} \tag{8}$$

Equation (8) of IoT-fog computing environments is critical for determining the processing time of tasks at the end nodes based on length of the tasks. Where $\sum_{i=1}^{n} EN_n$ represents the total process time for all end nodes, length is the total length of the tasks assigned to the end nodes, $\frac{2}{1000}$ is a scaling factor that converts the task length into processing time. This factor might be derived from specific system parameters such as the processing speed of the end nodes or empirical data. The primary purpose of this equation is to provide a way to translate the length of tasks into measurable process time, essential for scheduling and resource allocation. Accurate process time calculations help optimize task distribution, reduce waiting times, and enhance overall system performance

$$FN = \sum_{i=1}^{n} FN_n \tag{9}$$

This Eq. (9) presents total number of fog nodes. FN represents fog nodes, FN_n denotes fog nodes in the n-th category. Fog nodes play a significant role in processing tasks offloaded from the end nodes, thereby reducing latency and improving efficiency.

The task scheduler was used to assigned the user requests for tasks to the resource management for the task scheduling architecture shown Fig. 2. Although tasks are actually carried out in the cloud, the cloud has a few latency problems, thus the scheduler can be created in fog nodes. Based on task length and processing capabilities, the resource manager for the fog interface application gathers requests for each task that is submitted. Based on the recommendations of the scheduler, of the model DRLMOTS, tasks must be scheduled effectively onto the VMs. The scheduler to the execution queue must first send these priority jobs, and then they must be sent to the VMs. Our scheduler must monitor upcoming requests in our proposed architecture and notify the resource manager. Tasks are directly mapped to the cloud if they have a low processing time; otherwise, they go through the fog scheduler. Our suggested architecture keeps track of upcoming requests that will be executed, requests in virtual machines, and virtual resources in resource management for each time interval T scheduler. Therefore, based on these circumstances, the scheduler will make a dynamic choice, such as assigning a job to an existing VM or a new VM. If an existing VM has the storage and processing power to accept new activities, you can add them to it or move on-going work to other VMs. Using a method called DRLMOTS, we were able to organize the jobs in an intelligent manner based on the predetermine circumstances for each time interval T. It will be updating the scheduler with its scheduling choices and take proper care of scheduling tasks into VMs. This scheduler's main objective is to efficiently map jobs to virtual machines (VMs) depending on their priority while minimizing metrics like makespan, energy usage, and fault tolerance. In order to identify task dependencies, it is necessary to first assess work priorities. As a result, after determining task priorities, the overall load on the VMs must been calculated. Overall load balancing to fog VMs are can be identified the Eq. 10

$$lo^{VMq} = \sum lo^{n} \tag{10}$$

where lo^{n} represents the current state running n number of VMs and lo^{VMq} indicates the load on the VM queue. When both cloud and fog nodes have completed the workload, workload assigner must decide where to locate the workload. This situation arises when both the cloud and fog computing resources have processed a task, and a decision needs to be made about where the final processed data should reside. This results in the workload requesting access to the processed information in the cloud when more resources are needed [28]. This means that if a task requires additional resources that are not available in the fog node, it can request to access the resources and information available in the cloud. Therefore, a threshold is established and adjusted according to the current set of workloads to optimize the number of effective workloads on the node. This threshold acts as a limit or guideline to determine the resource allocation between the cloud and fog nodes. It is fine-tuned based on the current tasks to ensure that the maximum number of tasks are effectively processed by the fog nodes. The workload assigner needs to set the choice threshold $\delta_{j,p}$, for each time period, which is the resource threshold of node J within interval p. this threshold represents the maximum resources that node j can utilize during the time interval p. Initially, the workload assigner calculates the resources requirements $(Rejavg)$

for each workload period. This calculation involves determining the average resources needed for the workloads during each period. If the resource requirement exceeds $\delta_{j,p}$, the workload is sent to the cloud; otherwise, it is allocated to the node for execution. This means that if the resources needed for a task are more than the threshold $\delta_{j,p}$,

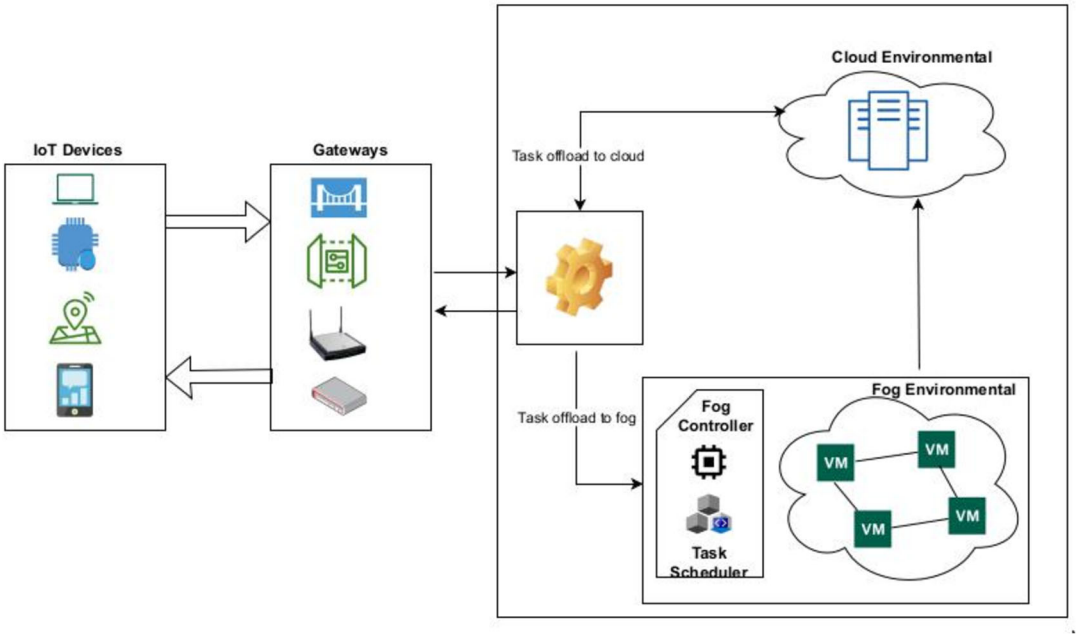

Fig. 2 Proposed task scheduling architecture

the task will be processed in the cloud. If the resources needed are within the threshold, the task will be processed in the fog node. If the number of workloads determined in p is greater than those in p – 1, the threshold for p is deemed suboptimal. Hence, the workload assigner reduces the threshold based on the average workload in p, represented as:

$$\delta_{j,p+1} = \begin{cases} \delta_{j,p}, if R_{j,p} > 1 and T_{j,p} < T_{j,p} 1 \\ max(Rejavg(t) | t \in T_{j,p}), if P > 1 and T_{j,p} \geq T_{j,p} 1 \end{cases} \tag{11}$$

$$\delta_{j,p+1} = \{R_{j,p}, ifp = 1 \tag{12}$$

In this equation, R_j represents the total resources of the node at any given interval. This variable indicates the overall capacity of node j during any period. During the threshold adjustment, the priority of each workload must be considered to determine the most efficient schedule for execution. This adjustment ensures that the most critical tasks are processed first, optimizing the scheduling process. Threshold as the fog computing was a dynamic and processing large number of requests coming to VMs. Therefore, if task-processing time was greater than the threshold value those tasks were allowed to fog otherwise allowed to cloud. The threshold value in this model can been calculated the following Eqs. (11), (12) high processing time tasks were going to fog node VMs low processing time tasks were going to cloud.

After this calculate load in virtual machines VMs, as all VMs run in fog nodes and physical host H_P. Therefore, overall load *lo* on hosts are calculate the Eq. (13)

$$\sum_{i=1}^{n} lo_{H_P}^{FN} = lo^{VMq} / \sum H_P \tag{13}$$

where $lo_{H_P}^{FN}$ represents the load on the P physical hosts of fog node VMs, lo^{VMq} indicates the current load on all VMs, H_P represents the physical hosts. If the selected fog nodes, VM queue is full or not it is full find another available VM within the fog node dynamically. Therefore, proposed model DRLMOTS can be calculated using following Eq. (14)

$$\exists f \epsilon F : \neg is_{full(Q(f))} \tag{14}$$

where 'F' represents the set of fog nodes, we're trying to find a fog node 'f' such that its VM queue is not full

$$Q(selected_{node}, new) = APPEND(Q(selected_{node}, old)T) \tag{15}$$

In Eq. (15) shows the 'T' represents the tasks, and 'APPENDIX' (Q, T) represents appending task 'T' to queue 'Q'. Also, let Q (selected node) represents the VM queue of the selected node after these tasks are fit into VMs. In this research work, focused to address the metrics execution time, energy consumption, fault tolerance in both cloud and fog environments. Makespan will be formulated in both cloud and fog we seen in Eq. (16)

$$Makespan(M) = max_{i=1}^{n} \sum_{j=1}^{mi} t_{ij} \tag{16}$$

where 'n' represents number of tasks in computing *mi* represents number VM machine and t_{ij} total processing

time of jobs and across all machines. Our next aim was to reduce energy consumption both cloud and fog computing. It is the most important and influential aspects in the cloud and fog paradigm [29, 30]. Processing massive workloads necessitates large-scale infrastructure of cloud resources, which increases energy consumption CO_2 emissions and environmental impact. As a result, we are concentrating on reducing energy consumption in both cloud and fog. Energy consumption is dependent on computing time and power consumption this process will done in VMs finally we evaluated the energy consumption we seen Eq. (17), (18), (19)

$$E = P \times T \tag{17}$$

$$E_{Transmission} = D \times E_{data} \tag{18}$$

$$E_{Total} = \frac{E + E_{Transmission}}{T} \tag{19}$$

Here P represents the power consumption, T represents the time it is active next the energy used for data transmission is computed by multiplying the amount of transmitted data the energy cost per unit of data. Our next focus was fault tolerance to overcome the failure events we used the backup function to minimizing the fault tolerance. The common approach to measuring the fault tolerance is use the concept of redundancy. The basic idea is to introduce redundancy in the system so if one node fails another can take over. The formula for fault tolerance in a redundant system can be expressed as Eq. (20)

$$FT = 1 - \prod_{i=1} n \tag{20}$$

The corresponding backup fog nodes for was denoted as, the fault tolerance of the system FT n is the number of redundant components

$$Faulttolerance(FT) = 1 - \prod_{i=1}^{n} P(F_i) \tag{21}$$

where fault tolerance was evaluated $P(F_i)$ is the probability of failure for the i^{th} component.

5 Methodology

In this section, it says how tasks are fed into our proposed approach DRLMOTS. The DRLMOTS is ML-based reinforcement learning technique enables agents to make decisions depending on the input supplied to the system. A machine-learning (ML) model consists of several states in which the agent performs certain behaviors depending on input, resulting in the production of rewards that may be either positive or negative. These rewards play a important role in guiding the algorithm's decision in subsequent states. The essence of the reinforcement learning approach lies agents learning from the rewards received prior states, aiding them in making informed decisions over time. The system learns from experiences, stands as a key advantage of the reinforcement learning approach. DRL for task scheduling in this context agent learns from the historical sequences of incoming user requests or tasks on the fog and cloud console. Initially a prioritized user request serves as input for agents, which then makes scheduling decisions based on the prevailing conditions in the fog environment. The agent's decision results in the execution of a task or user requests with the outcome (fault tolerance, energy consumption or fault tolerance) serving as a reward [31, 32]. If the reward was negative, the agent adjusted its decision making to parameters to enhance the performance. If the reward was positive, it stores this knowledge for future decision making in subsequent states shown in the Fig. 3

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha.\left[ret_t + \gamma.maxa_t^{\cdot} Q\left(s_t^{\cdot}, a_t^{\cdot}\right)\right] - Q(s, a) \tag{22}$$

In our scheduling model DRLMOTS, we employ Q-learning technique Q- learning makes a decision based on past actions which represented in pairs as Q- function for specific states and actions $Q(s_t, a_t)$. The model updates its states using its states using an equation involving a learning rate(α), controlling how much the Q- value is update in each iteration, an ($[ret_t]$) immediate rewards obtained after take the action(a_t) in states(s_t) and a discount factor (γ), which immediate rewards against future rewards. $s_t^{\cdot}$ Is the next state after taking action a_t in state s_t action $a_t^{\cdot}$, represents all possible actions in state $s_t^{\cdot}$, and $maxa_t^{\cdot} Q(s_t^{\cdot}, a_t^{\cdot})$ calculate the maximum Q-value for those actions in the next state. This iterative process involves the model assessing rewards and adjusting its decisions accordingly. However, applying classical Q- learning to np-hard problems like fog and cloud computing scheduling with a high number of states and actions, challenges due to the vast state-action space [33]. To address this, we integrate the deep neural networks with reinforcement learning to create a deep reinforcement-learning model. This model is well suited for tackling the fog and cloud computing scheduling issues. The integration of reinforcement learning into various scheduling techniques has been successful as evidence of the existing work [16]. we use a deep neural network in our method to improved scheduling in fog layers and cloud computing. The goal of strategy develops an intelligent scheduler capable of making decisions in real time without knowing the outcome, which would then be provided to the agent and algorithm for decision-making. The point when constant information was given as information. Like q realizing, which comprises of state, activity, award, and target, it comprises of different states.

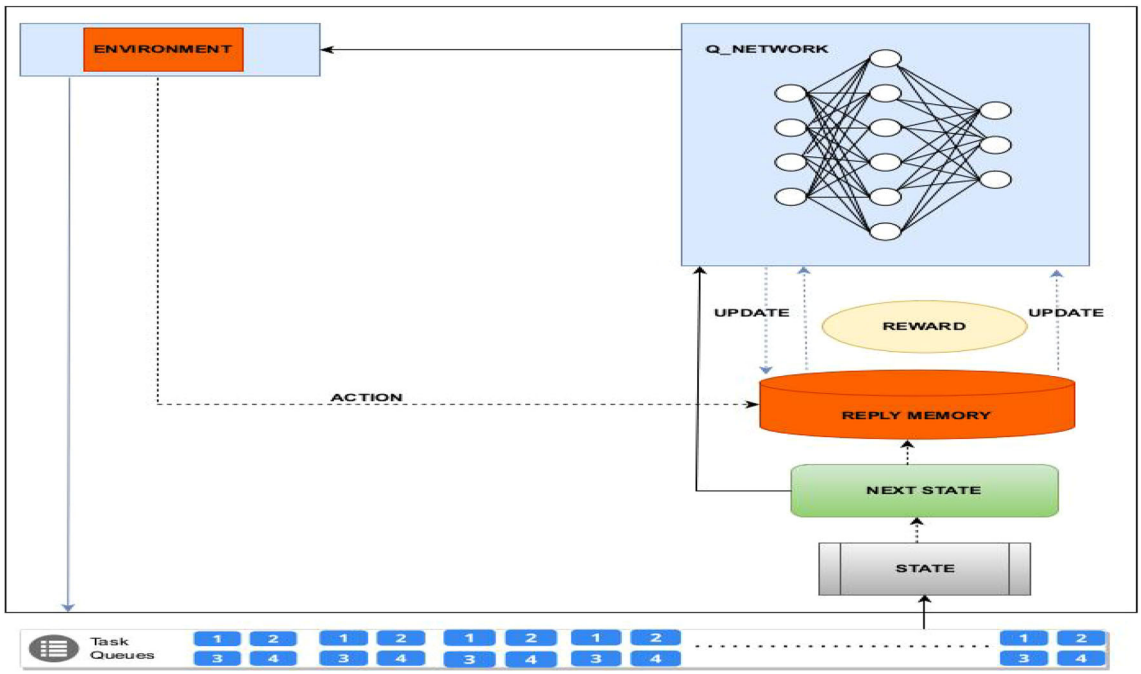

Fig. 3 Working of DRLMOTS Scheduler

5.1 State space

The state space is defined to capture the current status of the fog-cloud environment, including the number of tasks, resource availability at fog and cloud nodes, and network conditions. Each state provides a comprehensive view of the system, enabling the algorithm to make informed decisions.

$$s_t = s_t^{;} \cup s^{vm} \tag{23}$$

where s_t a state of a t is task at time T and s^{vm} is a state of vm_n when t task comes on to vm at T time.

5.2 Action space

We previously mention, VMs were taken consideration in this study in action space. All incoming requests are initially routed through the task manager, and each task priority is then determined and provided to the scheduler, which uses the DRLMOTS model to make decisions about which tasks to submit to the execution queue based on their priority. The tasks must then be entered into the queue according to their priorities and executed on the VMs based on the Scheduler's decision. As a result, our model's definition of action space is as follows.

$$\mathrm{A} = \{vm_1, vm_2, vm_3, \ldots\ldots\ldots vm_n\} \tag{24}$$

5.3 Reward function

Reward functions are used to guide the learning process by providing feedback on the quality of the agent action. The basic goal of reinforcement learning rewards is to assist the agent in learning to make decisions that lead to the attainment of its objectives or goals. The reward (ret) is a numerical value that offers feedback to the agent, reflecting the quality of the current state's action. It has an impact on the agent's learning process and execution time of the task offloaded to a fog node. It is assigned to a negative value to encourage the agent to run as quickly as possible to minimize the metrics makespan, fault tolerance, energy computation. It is expressed using Eq. (25)

$$ret = min(m^t, fault_tolerance^t, E^t) \tag{25}$$

Input: Number of tasks $t_n = \{t_1, t_2, t_3, \ldots\ldots\ldots\ldots\ldots t_{k-1}\}$ number of VMs $VM_n = \{vm_1, vm_2, vm_3, \ldots\ldots\ldots\ldots vm_{n-1}\}$

Number of fog nodes = $\{FN_1, FN_2, FN_3, \ldots\ldots\ldots\ldots\ldots VM_{k-1}\}$

and scheduling parameters.

Output: Mapping of the tasks to virtual resource while minimizing the energy consumption, makespan, fault tolerance.

Algorithm 1 Algorithm of DRLMOTS

1. Initialize the t_n,FN_n,$clouyd_n$
2. Initialize the parameters state, action, replay buffer (s,a,ret,s')// **where 's' is the** current **state 'a' is the action taken 'ret' is the reward received, 's'' is the next state**
   2.1. $\gamma = 0.95$ //**Discount factor**
   2.2. $\epsilon = 1.0$ //**The exploration rate initial set to 0.1 for full exploration**
   2.3. $\epsilon_{min} = 0.01$ //**The minimum exploration rates**
   2.4. $\epsilon_{decay} = 0.01$ //**The decay rate for exploration rate**
3. Network model building
   3.1. $Q(s,a,\theta)$ // **where θ represents the weight**
      3.1.1. **Input layer** {Input layer}
      3.1.2. $h_1 = ReLu(Dense(s,w_1,b_1))$
      3.1.3. $h_2 = ReLu(Dense(h_2,w_2,b_2))$
      3.1.4. **Output layer :** $Q(s,a,\theta) = Dense(h_2,w_2,b_2)$
   3.2. $\theta = w_1,b_1,w_2,b_2,w_3,b_3$ // **The weight and bias of neural network**
4. // **Q-learning target update**

$$Q(s,a,\theta) = \begin{cases} ret \rightarrow if\ episode\ is\ done(done = True) \\ ret\ +\ \gamma.max_a Q\left(s',a,\theta_{target}\right) \rightarrow if\ episode\ is\ not\ done \\ (done = False) \end{cases}$$

5. $memory(\omega) = (s,a,ret,\ s',done)$// **The replay buffer stores the collection of tuples**
6. **The agent selects action using an epsilon greedy policy**

$$\pi\left(\frac{a}{s}\right) = \begin{cases} random\ action\ with\ probability\ \epsilon \\ arg\ max_a\ Q(s,a,\theta)\ with\ probability\ 1 - \epsilon \end{cases}$$

7. $update\ the\ targetmodel$
8. $repeat\ the\ targetmodel$

5.4 Training agent

The incoming tasks arrived at the scheduler the DRLMOTS agent must make a choice in the present state by evaluating task priority to resources, and then map the tasks to VMs. To do this our DRLMOTS model must be trained in such a way that it first maps the job to VMs by considering the conditions with the probability and its value decreases over time. As a result, the agent first investigates randomly and makes a decision, and then it makes a decision based on past Q-values in the q-tables. To do this experience buffer replay and fixed q-target values will be used the algorithms when an agent makes judgements over a period of time. It will earn experience through time, which will be reflected as experiences replay in our work. When an agent makes a decision, it pays you reward, which is represented as ret, and the next state is represented as s. These encounters will be kept as values in $memory(\omega)$ that will be represented as replay data storage denoted. The variables to be saved in this replay the $memory(\omega) = (s, a, ret, s'$,done) Whenever iterations are executed and replay memory values are to be updated. In our study, we added 50 neurons that were used in the model hidden layers. We kept the scheduling time for making a choice for an agent at 10 ms, the frequency of the learning at one.

The effectiveness and increased efficiency of the task scheduling process in cloud-fog computing environments are the main benefits resulting from the integration of the DQN into the DRLMOTS scheduler. Therefore, schedule of the tasks is a significant challenge that such tasks are diverse and the performance of fog and cloud computing resources differ significantly. The above challenge is solved through using the DQN model which identifies the most suitable computational resources to assign specific tasks considering their features and the state of the system. The DQN works by keeping track of a complete state which describes the current environment at any point of time with aspects such as the number of active tasks, the attributes of these tasks, the computational capacity at the fog cloud levels and real-time information from the actual network. Due to this detailed state representation the DQN is able to determine the best strategy to allocation of that tasks. Thus, it assigns tasks in the available resources with negligible time delays. In the action space, the DQN model selects the actions which tasks will be anticipated and handled by specific virtual machine (VM) or node in the fog cloud infrastructure. Every action corresponds to an evaluation on whether to execute a certain VM on a fog node. If the task can be accomplished quickly, or to use a VM on the cloud node if the task requires high computation power. The model is to make better decisions regarding those actions by evaluating the changes in general

performance of the system due to the individual actions. The reward function is proposed as an important component when integrating DQN by the fact that it helps the learning process by informing the system about the consequences of action performed. The choice of the reward function is as follows, that it is specialized in avoiding several key metrics makespan, energy, fault. By providing such objectives the DQN model trains itself to choose scheduling decisions that create optimum benefits using available resources while enhancing the general performance of the system. The learning process that takes place in the DQN utilization of experiences of past task arrival rates and the system performance results in the update of the DQN. This means that the action value or Q-values, which gives the expected value of a given action in a state, are modified according to the previously obtained rewards. It offers the possibility of learning during the process and modify that approach for scheduling the problem to be solved in the DQN, leading to better decision making. By implementing deep reinforcement learning, the DQN is capable of working with situations which involve high dimensions for both states and actions which is a challenge that usual scheduling algorithms are unsuitable for cloud-fog architectures. The DQN enhances the functionality of DRLMOTS scheduler to provide real time responses to the alteration of the workloads and system environment. The DQN determines the best action to perform when a new task occurs and when new system environment. The DQN determines the best action to perform when a new task occurs and when new system states appear, thus constantly improve the functionalities of assigning tasks and managing resources. This dynamic decision making is especially important in the complex and highly varying cloud-fog architectures where computation loads and resources vary in a short time. The research work presented in this paper the integration of DQN into the DRLMOTS model marks a substantial development in the domain of task scheduling for cloud -fog computing. The deep reinforcement learning the scheduler can make appropriate decisions in real-time as to improve the typical performance indicators which in turn improves the efficiency, actual capability, and flexibility of the cloud fog system. The proposed algorithm flow is explained in Fig. 4. All parameters, including replay memory, batch size, learning rate, epsilon and discount factor Fig. 4. The q-function state and action space then initialized to 0. In the following stage, the priority of tasks is calculated for each event for each incoming tasks to the fog and cloud platform. For each event, the scheduler must select an action, VMs for the matching state, tasks, depending on the task priority and resource availability on VMs. Every time the scheduler must make a judgment based on this circumstance. The following jobs will be assigned to associate VMs with a random probability based on their priority. If the initial task to be scheduled the DRLMOTS must make a decision for the existing q-table to the agent when an action space A is selected for each state and reward is generated. In our case, it is the minimizing metrics such as makespan, energy consumption, and fault tolerance Eq. (25) the reward value computed. On the off chance that it is a positive reward score, reward score will be improved on the off chance that it is a negative reward, the scheduler should work on in view of its insight. Positive prizes, for example, makespan, energy utilization, and adaptation to internal failure will be refreshed as limiting the upsides of present state. They are either negative or positive prizes will be kept in replay memory. In the wake of considering rewards, it ought to utilize Eq. (23) to refresh its state to the following state. This process is repeated until the last state, the last task is encountered Fig. 4

The proposed algorithm effectively handles the heterogeneity in resources by considering the specific characteristics and capabilities of fog and cloud nodes. For instance, real-time video processing tasks, which require low latency and moderate computational power, are assigned to fog nodes with sufficient computational resources and minimal latency, ensuring real-time processing. Data analytics tasks, involving large datasets and significant computational power, are directed to cloud nodes with abundant resources, allowing for efficient processing despite higher latency. IoT sensor data, which needs to be aggregated and processed in real-time, is assigned to nearby fog nodes to minimize latency and ensure timely processing. Batch processing tasks, such as database updates or large-scale simulations, which can tolerate higher latency, are allocated to cloud nodes, leveraging their high computational capacity to handle the intensive processing requirements. Through these strategic assignments, the algorithm optimizes task scheduling based on the specific needs and resource availability, enhancing overall system performance and efficiency.

6 Simulation and results

Section 6 presents results of proposed DRLMOTS The (SimPy) python-based discrete event simulation framework was used for all simulation work [22]. The simulation environment resembles the scene in Fig. 1, which depicts the state of our finished network. Four APs, four ENs, four FNs, four IoT devices, and a cloud datacenter were utilized. Fog nodes include FN1, FN2, FN3, FN4, and the datacenter. Either each computing node processes the task, or it is sent to another location. We allocate a set of virtual machines (VMs) to each fog node in order to best utilize the limited resources. The data size for each task varies

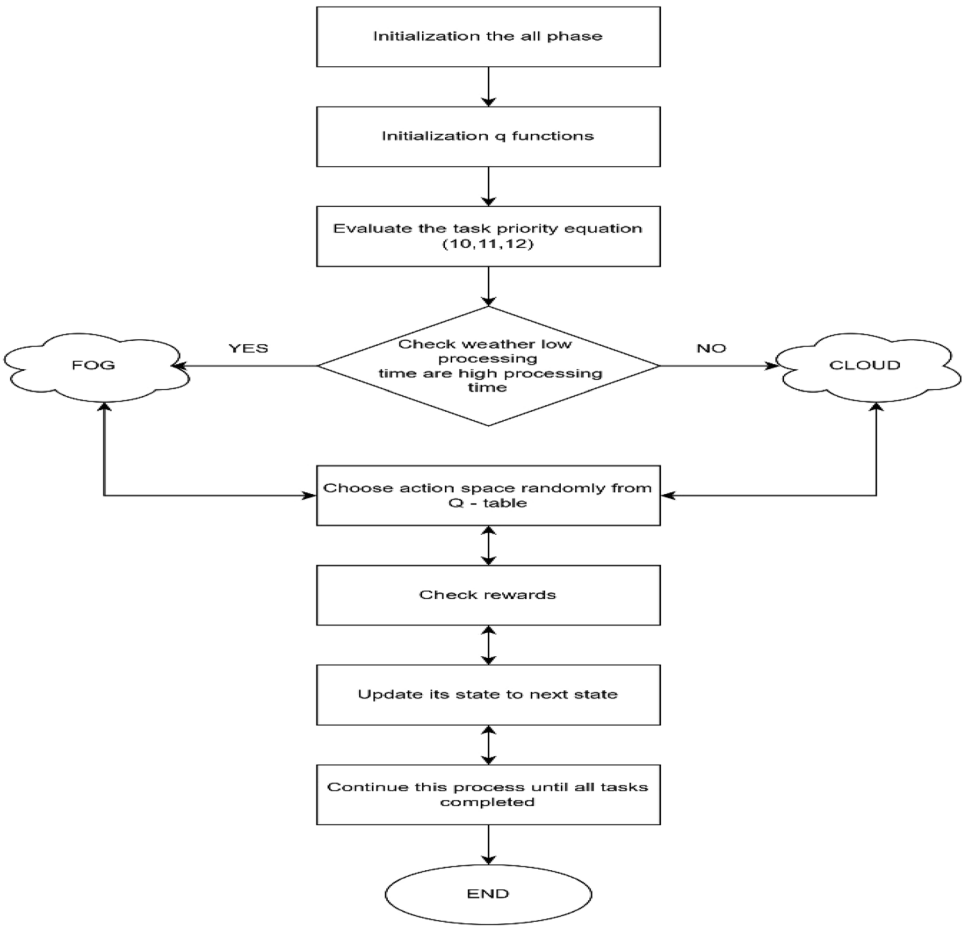

Fig. 4 Flow chart of proposed DRLMOTS

from 20 to 150kB. Each episode's task set, which consists of between 300 and 3000 jobs (generated by an IoT device), is randomly mixed up by schedulers. We calculated 150 episodes and iterations to test the suggested method over a range of hypermeter values. Milliseconds are used as the execution time unit, and tasks are generated at random intervals. We tested our DRLMOTS model against the existing baseline algorithms long short-term memory (LSTM), gated graph convolution network (GGCN) and convolution neural network (CNN) after providing these tasks as input to the algorithm. The selection of SimPy and the other fundamental libraries and modules enabled modeling of correct cloud-fog computing environments. The simulation setup, it was designed to resemble a real computational environment and provided such parameters as processor type (Intel i7) RAM size (32 GB), the amount of storage and network throughput. The network configuration involved not only the several fog nodes and cloud nodes but also provided IoT devices different roles in performing tasks depending on complexity calculation on computational abilities and latency. The task which was created for the simulation ranged in size and complexity of the computations required from low to high, and the tasks were randomly generated based on real datasets such as the google jobs datasets [34]. To determine the effectiveness of the scheduler under different circumstances these datasets were again divided into small, medium, and large depending upon their magnitude. Our results were compared against the baseline algorithms CNN, LSTM and GGCN. The purpose of testing the configuration that were taken into consideration while performing simulations there were arrangements in which the number of the virtual machines (VMs) was maintained constant while the number of the tasks was changed and vice versa to check the scalability and flexibility of the scheduler. The makespan, energy and effective fault-tolerant have been measured during the simulation which are the most important parameters that indicate how effective the task scheduling can be in context of cloud-fog systems. Some other metrics were developed in order to capture the scheduling efficiency in terms of handling the computation resources and degree of interruption. The simulation runs were made for the number of episodes (100) to achieve certain statistical significance of results. Each run was carried out in a very precise manner to examine the ability of the DRLMOTS scheduler in adapting and modifying

task assignments in response to the current state of the network setting. This was done in order to evaluate the performance of DRLMOTS against the baseline algorithms.

6.1 Calculation of makespan

Makespan is the general measure of time expected to get done with every one of the responsibilities in a scheduler. It essentially affects the frameworks in general viability, making it a pivotal measure. Achieving high throughput and responsiveness in applications requires minimizing makespan. Effective scheduling guarantees that resources are used to their fullest potential, cutting down on downtime and increasing output. We evaluated the makespan in two cases using configuration settings available in Table 3 and different tasks are given to our proposed model DRLMOTS. Initially we have given different workloads from google cloud jobs [34]. This dataset consists of 1000 tasks and remaining tasks are generated randomly by fabricating the dataset and divided into types of datasets small, medium and large. Small datasets consist of 300 to 1500 tasks, medium datasets consist of 1800 to 3000 tasks and large datasets consist of 3300 to 4500 tasks. Our testing results categorized the two cases.

6.2 Case 1

In case 1 tested for fixed VMs and randomly increases the tasks shown in Tables 4, 5, 6. Table 4 represents the makespan of small dataset for this we plot a result shown in Fig. 5.The proposed model DRLMOTS was evaluated against existing baseline algorithms: Convolutional neural network (CNN), long short-term memory (LSTM), and gated graph convolution network (GGCN). DRLMOTS ran for 100 iterations. Generated makespan for 300, 600, 900, 1200, 1500 tasks for CNN algorithm are 1012.5, 1307.6, 1619.3, 2072.5, 2531.8 respectively. Generated makespan for 300, 600, 900, 1200, 1500 tasks for LSTM algorithm are 882.7, 1188.8, 1540.3, 1953.9, 2413.3 respectively. Generated makespan for 300, 600, 900, 1200, 1500 tasks for GGCN algorithm are 778.7, 1084.8, 1396.3, 1849.9, 2309.7respectively. Generated makespan for 300, 600, 900, 1200, 1500 tasks for DRLMOTS are 687.1, 993.2, 1304.7, 1758.3, 2217.4 respectively. Table 5 represents the makespan of medium dataset for this result shown in the Fig. 6. The proposed model DRLMOTS was evaluated against existing baseline algorithms: Convolution neural network (CNN), long short-term memory (LSTM) and gated graph convolution network (GGCN). DRLMOTS ran for 100 iterations. Generated makespan for 1800, 2100, 2400, 2700, 3000 tasks for CNN algorithm are 3133.4, 3837.8, 4410.2, 5495.6, 6016.4respectively. Generated makespan for 1800, 2100, 2400, 2700, 3000 tasks for LSTM algorithm 3014.3, 3718.4, 4292.3, 5376.4, 5897.7 are respectively. Generated makespan for 1800, 2100, 2400, 2700,

Table 3 Simulation settings

Entities	Quantity
Total number of tasks	300–3000
Task lengths	700,000
Storage of Host	3 TB
Host RAM	32 GB
Bandwidth	1000MBPS
Number of VMs	20—200
Bandwidth of VMs	200MBPS
Processor	Intel(i7)
Operating System	Linux
Bandwidth of VMs	200MBPS

Table 4 evaluation of makespan for small dataset

Tasks	CNN	LSTM	GGCN	DRLMOTS
300	1012.5	882.7	778.7	687.1
600	1307.6	1188.8	1084.8	993.2
900	1619.3	1540.3	1396.3	1304.7
1200	2072.5	1953.9	1849.9	1758.3
1500	2531.8	2413.3	2309.7	2217.4

Table 5 Evaluation of makespan for medium dataset

Tasks	CNN	LSTM	GGCN	DRLMOTS
1800	3133.4	3014.3	2910.3	2818.7
2100	3837.8	3718.4	3614.4	3522.8
2400	4410.2	4292.3	4188.1	4096.5
2700	5495.6	5376.4	5272.2	5180.6
3000	6016.4	5897.7	5793.5	5701.9

Table 6 Evaluation of makespan for large dataset

Tasks	CNN	LSTM	GGCN	DRLMOTS
3300	6532.3	6413.5	6309.5	6217.9
3600	7082.6	6963.9	6859.3	6768.3
3900	7324.3	7205.4	7101.4	7009.9
4200	7665.4	7547.2	7443.2	7351.4
4500	8052.7	7933.8	7829.4	7738.2

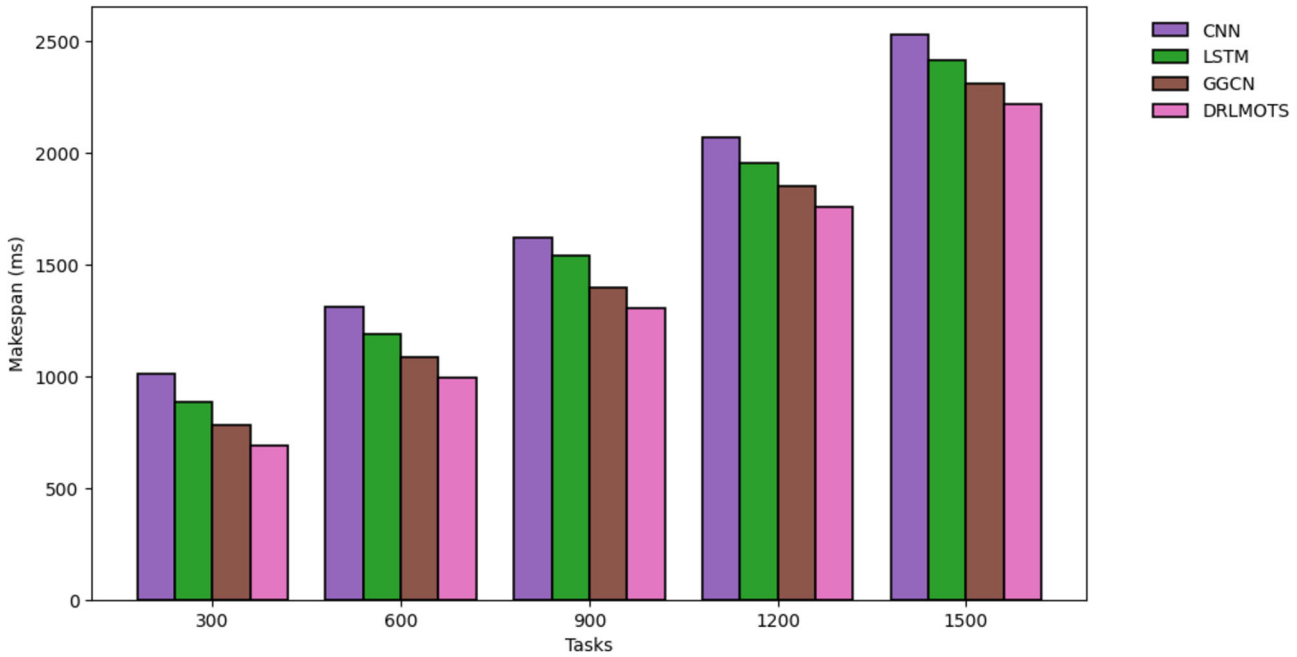

Fig. 5 Makespan of DRLMOTS using fixed VMs for small dataset

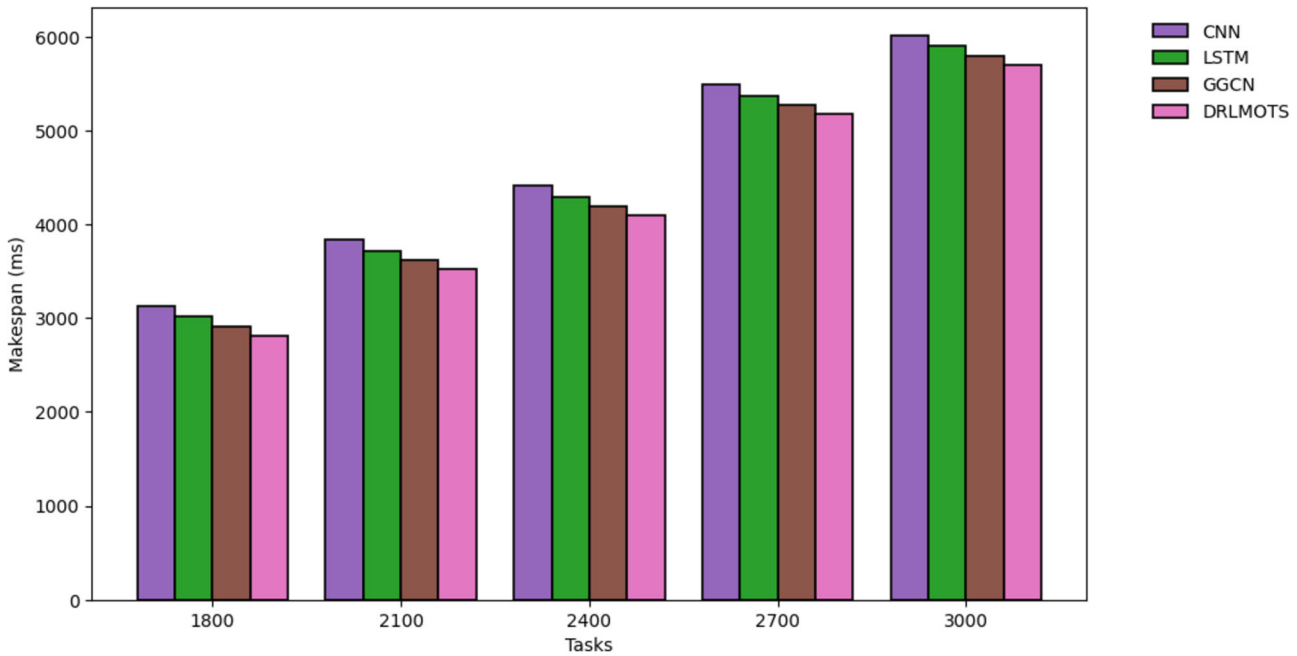

Fig. 6 Makespan of DRLMOTS using fixed VMs for medium dataset

3000 tasks for GGCN algorithm are 2910.3, 3614.4, 4188.1, 5272.2, 5793.5respectively. Generated makespan for 1800, 2100, 2400, 2700, 3000 tasks for DRLMOTS are 2818.7, 3522.8, 4096.5, 5180.6, 5701.9 respectively. Table 6 represents the makespan of large dataset for this result was shown in the Fig. 7 The proposed model DRLMOTS was evaluated against existing baseline algorithms: Convolution neural network (CNN), long short-term memory (LSTM),and gated graph convolution network (GGCN). DRLMOTS ran for 100 iterations. Generated makespan for 3300, 3600, 3900, 4200, 4500 tasks for CNN algorithm are 6532.3, 7082.6, 7324.3, 7665.4, 8052.7respectively. Generated makespan for 3300, 3600, 3900, 4200, 4500 tasks for LSTM algorithm are 6413.5, 6963.9, 7205.4, 7547.2, 7933.8respectively. Generated makespan for 3300, 3600, 3900, 4200, 4500 tasks for

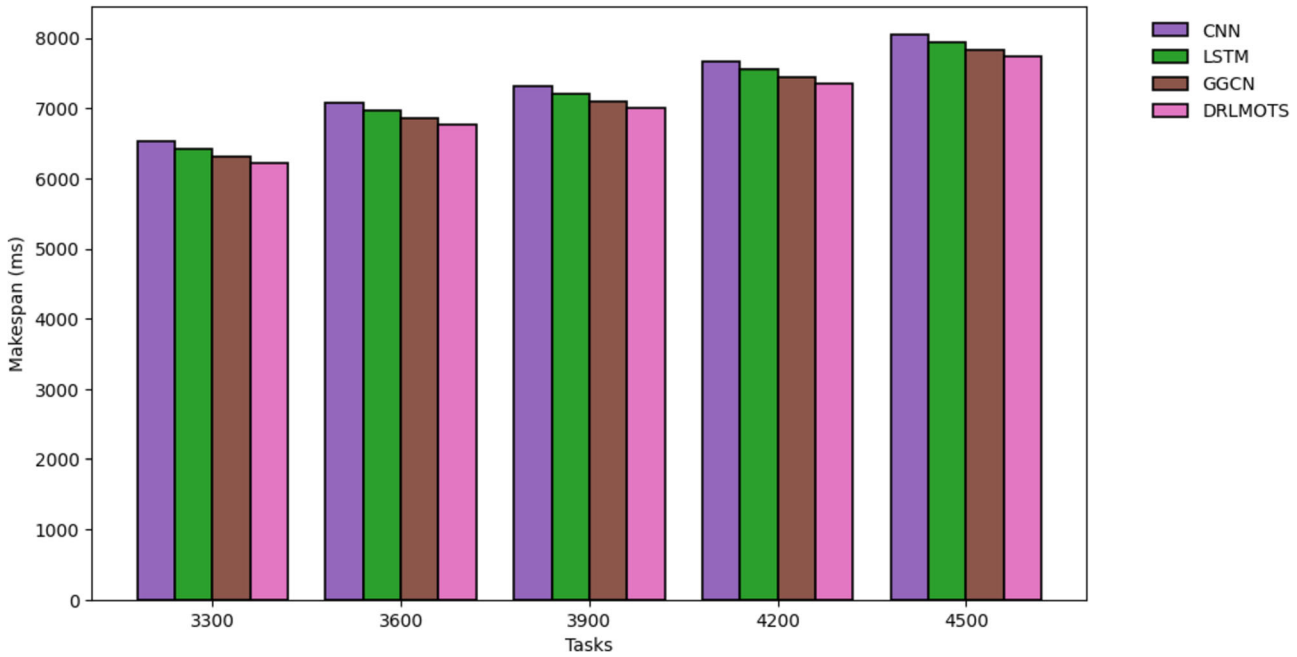

Fig. 7 Makespan of DRLMOTS using fixed VMs for large dataset

GGCN algorithm are 6309.5, 6859.3, 7101.4, 7443.2, 7829.4respectively. Generated makespan for 3300, 3600, 3900, 4200, 4500 tasks for DRLMOTS 6217.9, 6768.3, 7009.9, 7351.4, 7738.2 are respectively. Below Tables indicates calculation of makespan. The tables below clearly indicate that our suggested model DRLMOTS method assessed over CNN, LSTM, and GGCN algorithms, and from simulation results, makespan is minimizing over above approaches.

6.3 Case 2

In case 2 evaluated the makespan into linearly increases the VMs and randomly increases the tasks shown in Table 7, 8, 9. Table 7 represents the makespan of small dataset for this we plot a result shown in Fig. 8.The proposed model DRLMOTS was evaluated against existing baseline algorithms: Convolution neural network (CNN), long short-term memory (LSTM), and gated graph convolution network (GGCN). DRLMOTS ran for 100 iterations.

Table 7 Evaluation of makespan for small dataset

Tasks	CNN	LSTM	GGCN	DRLMOTS
300	977.6	879.3	775.3	692.4
600	982.4	885.3	781.4	694.2
900	988.3	889.7	789.6	697.3
1200	992.4	892.3	794.5	698.8
1500	998.7	898.4	797.2	669.7

Table 8 Evaluation of makespan for medium dataset

Tasks	CNN	LSTM	GGCN	DRLMOTS
1800	983.6	864.8	760.8	669.2
2100	988.7	870.1	766.1	674.5
2400	984.5	865.7	761.8	670.1
2700	987.6	869.3	765.3	673.5
3000	975.6	856.8	752.8	665.2

Table 9 Evaluation of makespan for large dataset

Tasks	CNN	LSTM	GGCN	DRLMOTS
3300	976.5	858.1	754.3	662.5
3600	983.7	864.4	760.8	669.3
3900	980.8	862.3	758.2	666.5
4200	979.5	860.4	756.2	664.6
4500	978.3	859.5	755.3	663.9

Generated makespan for 300, 600, 900, 1200, 1500 tasks for CNN algorithm are 977.6, 982.4, 988.3, 992.4, 998.7respectively. Generated makespan for 300, 600, 900, 1200, 1500 tasks for LSTM algorithm are 879.3, 885.3, 889.7, 892.3, 898.4 respectively. Generated makespan for 300, 600, 900, 1200, 1500 tasks for GGCN algorithm are 775.3, 781.4, 789.6, 794.5, 797.2respectively. Generated makespan for 300, 600, 900, 1200, 1500 tasks for DRLMOTS are 692.4, 694.2, 697.3, 698.8, 669.7

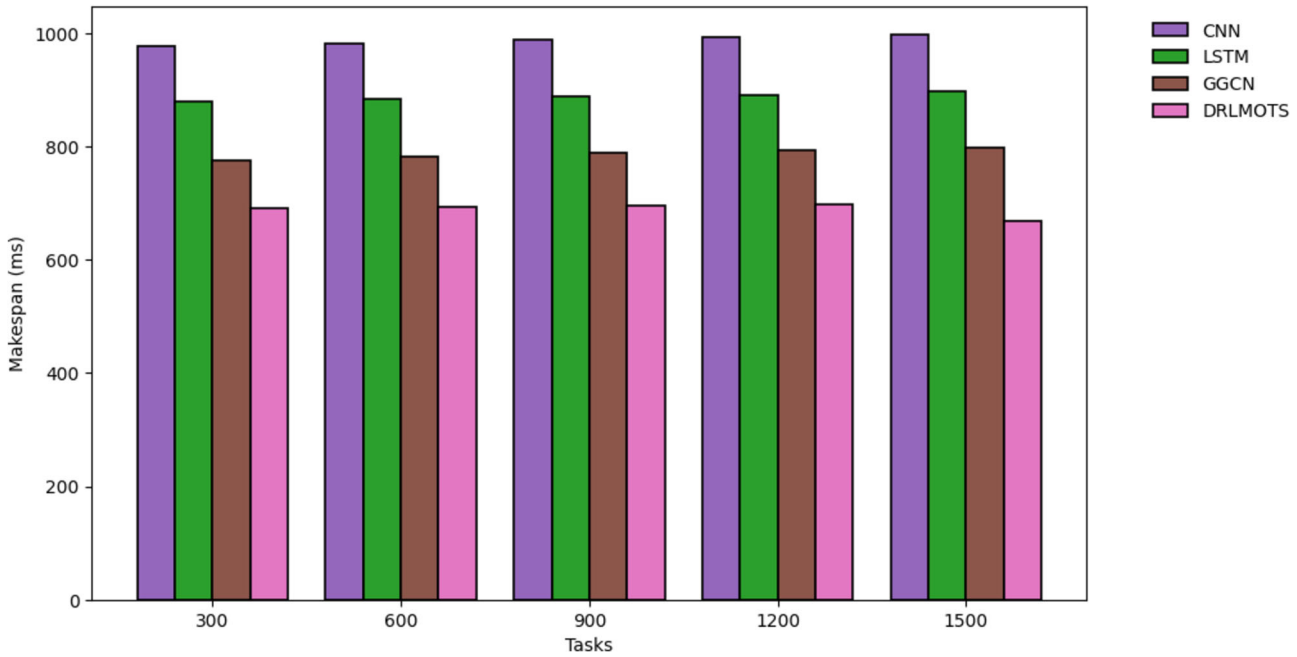

Fig. 8 Makespan of DRLMOTS using linear VMs for small dataset

respectively. Table 8 represents the makespan of medium dataset for this result shown in the Fig. 9 The proposed model DRLMOTS was evaluated against existing baseline algorithms: Convolution neural network (CNN), and long short-term memory (LSTM), gated graph convolution network (GGCN). DRLMOTS ran for 100 iterations. Generated makespan 1800, 2100, 2400, 2700, 3000 tasks for CNN algorithm are 983.6, 988.7, 984.5, 987.6, 975.6respectively. Generated makespan for 1800, 2100, 2400, 2700, 3000 tasks for LSTM algorithm are 864.8, 870.1, 865.7, 869.3, 856.8respectively. Generated makespan for 1800, 2100, 2400, 2700, 3000 tasks for GGCN algorithm are 760.8, 766.1, 761.8, 765.3, 752.8 respectively. Generated makespan for 1800, 2100, 2400, 2700, 3000 tasks for DRLMOTS are 669.2, 674.5, 670.1, 673.5, 665.2 respectively. Table 9 represents the makespan large

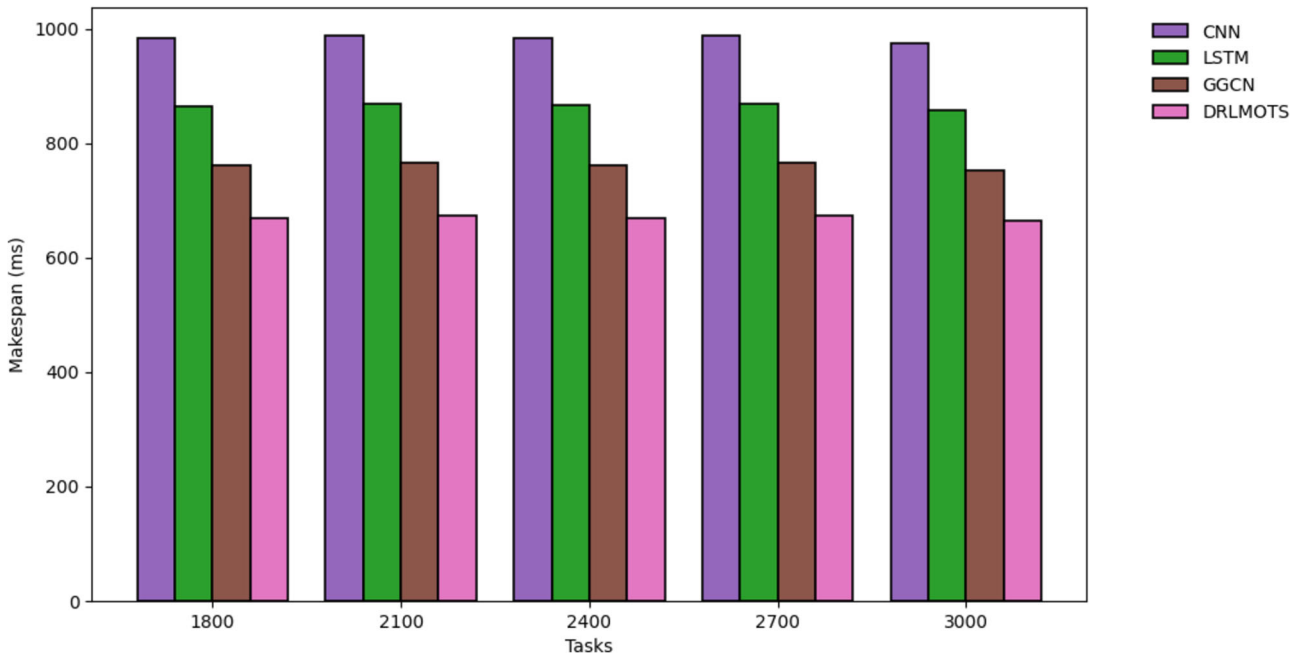

Fig. 9 Makespan of DRLMOTS using linear VMs for medium dataset

dataset for this result was shown in the Fig. 10. The proposed model DRLMOTS was evaluated against existing baseline algorithms: Convolution neural network (CNN), and long short-term memory (LSTM), gated graph convolution network (GGCN). DRLMOTS ran for 100 iterations. Generated makespan for 3300, 3600, 3900, 4200, 4500tasks for CNN algorithm 976.5, 983.7, 980.8, 979.5, 978.3respectively. Generated makespan for 3300, 3600, 3900, 4200, 4500 tasks for LSTM algorithm are 858.1, 864.4, 862.3, 860.4, 859.5respectively. Generated makespan for 3300, 3600, 3900, 4200, 4500 tasks for GGCN algorithm 754.3, 760.8, 758.2, 756.2, 755.3 are respectively. Generated makespan for 3300, 3600, 3900, 4200, 4500tasks for DRLMOTS 662.5, 669.3, 666.5, 664.6, 663.9 are respectively. The following tables show the computation of makespan. The tables clearly demonstrate that our suggested DRLMOTS model approach assessed over CNN, LSTM, and GGCN calculations, and from simulation results, makespan is minimizing over mentioned calculations.

6.4 Calculation of energy consumption

The energy consumption used by the system to carry out a task is referred to as energy consumption. It is an important factor to take into account, especially in contexts that use batteries or limited energy. Lowering energy consumption helps computer systems have a smaller negative environmental impact. Energy consumption was evaluated by using the Table 3 configuration settings and different tasks were given to DRLMOTS The workloads are taken from by the google job datasets Tables 10, 11, 12. Table 10 represents the energy consumption of small dataset for this we plot a result shown in Fig. 11. The proposed model DRLMOTS was evaluated against existing baseline algorithms: Convolution neural network (CNN), and long short-term memory (LSTM), gated graph convolution network (GGCN) DRLMOTS ran for 100 iterations. Generated energy consumption for 300, 600, 900, 1200, 1500 tasks for CNN algorithm 121.8, 163.2, 202.9, 249.1, 298.4 are respectively. Generated energy consumption for 300, 600, 900, 1200, 1500 tasks for LSTM algorithm 98.23, 139.5, 188.3, 227.9, 276.7 are respectively. Generated energy consumption for 300, 600, 900, 1200, 1500 tasks for GGCN algorithm 79.71, 121.2, 163.7, 202.5, 249.2 are respectively. Generated energy consumption for 300, 600, 900, 1200, 1500 tasks for DRLMOTS 64.28, 98.16, 124.7, 146.9, 173.4are respectively. Table 11 represents the energy consumption of medium dataset for this result shown in the Fig. 12. The proposed model DRLMOTS was evaluated against existing baseline algorithms: Convolution neural network (CNN), and long short-term memory (LSTM), gated graph convolution network (GGCN) DRLMOTS ran for 100 iterations. Generated energy consumption for 1800, 2100, 2400, 2700, 3000 tasks for CNN algorithm 339.2, 381.4, 421.2, 469.3, 510.1 are respectively. Generated energy consumption for 1800, 2100, 2400, 2700, 3000 tasks for LSTM algorithm 318.9, 365.6, 406.4, 445.2, 494.4 are respectively. Generated energy consumption for 1800, 2100, 2400, 2700, 3000 tasks for

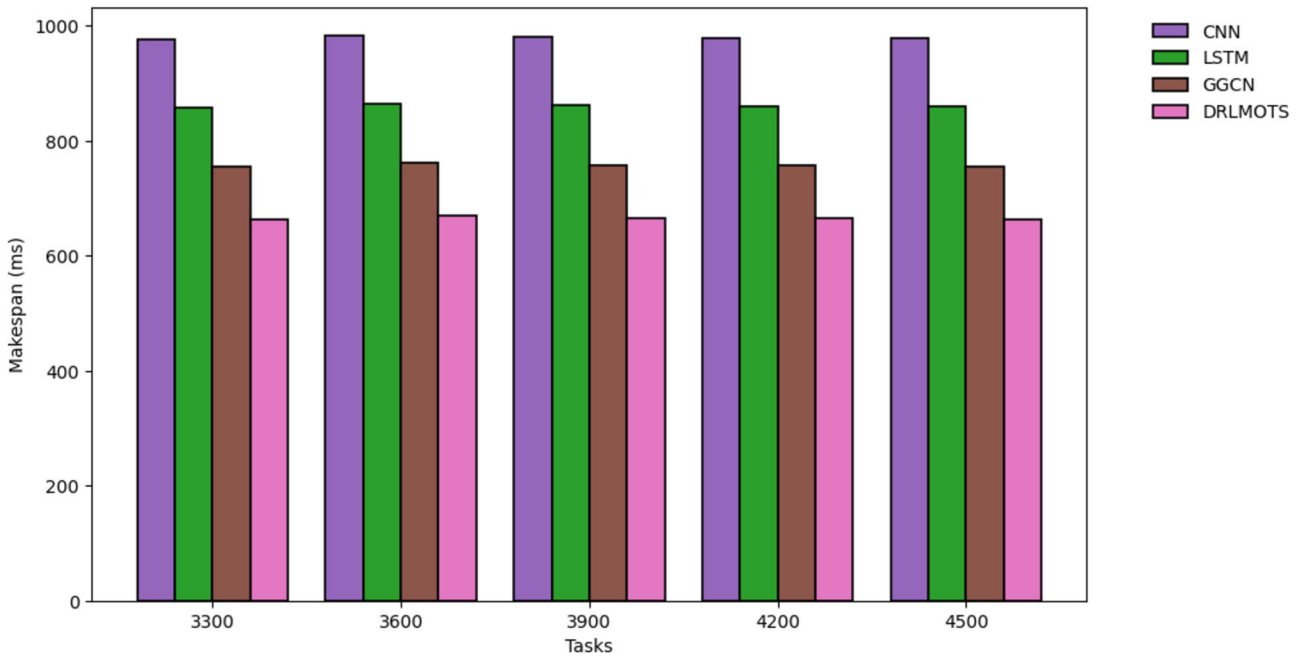

Fig. 10 Makespan of DRLMOTS using linear VMs for large dataset

Table 10 Evaluation of energy consumption for small dataset

Tasks	CNN	LSTM	GGCN	DRLMOTS
300	121.8	98.23	79.71	64.28
600	163.2	139.5	121.2	98.16
900	202.9	188.3	163.7	124.7
1200	249.1	227.9	202.5	146.9
1500	298.4	276.7	249.2	173.4

Table 11 Evaluation of energy consumption for medium dataset

Tasks	CNN	LSTM	GGCN	DRLMOTS
1800	339.2	318.9	298.2	196.9
2100	381.4	365.6	339.5	219.5
2400	421.2	406.4	378.8	246.3
2700	469.3	445.2	425.6	288.2
3000	510.1	494.4	468.2	314.5

Table 12 Evaluation of energy consumption for large dataset

Tasks	CNN	LSTM	GGCN	DRLMOTS
3300	562.9	543.2	508.4	346.7
3600	611.3	598.8	555.1	398.1
3900	658.2	639.6	594.9	425.6
4200	721.1	696.7	635.3	463.7
4500	794.2	723.1	698.7	489.3

GGCN algorithm 298.2, 339.5, 378.8, 425.6, 468.2 are respectively. Generated energy consumption for 1800, 2100, 2400, 2700, 3000 tasks for DRLMOTS 196.9, 219.5, 246.3, 288.2, 314.5 are respectively. Table 12 represents the energy consumption of large dataset for this result was shown in the Fig. 13. The proposed model DRLMOTS was evaluated against existing baseline algorithms: Convolution neural network (CNN), and long short-term memory (LSTM), gated graph convolution network (GGCN) DRLMOTS ran for 100 iterations. Generated energy consumption for 3300, 3600, 3900, 4200, 4500 tasks for CNN algorithm are 562.9, 611.3, 658.2, 721.1, 794.2 respectively. Generated energy consumption for 3300, 3600, 3900, 4200, 4500 tasks for LSTM algorithm are 543.2, 598.8, 639.6, 696.7, 723.1 respectively. Generated energy consumption for 3300, 3600, 3900, 4200, 4500 tasks for GGCN algorithm are 508.4, 555.1, 594.9, 635.3, 698.7 respectively. Generated energy consumption for 3300, 3600, 3900, 4200, 4500 tasks for DRLMOTS are 346.7, 398.1, 425.6, 463.7, 489.3 respectively. Below Tables indicates evaluation of energy consumption. The following tables clearly illustrate our suggested DRLMOTS model method assessed over CNN, LSTM, and GGCN algorithms, and from simulated results of energy usage is minimizing over the above algorithms.

6.5 Calculation of fault tolerance

Fault tolerance was evaluated by using the Table 3 configuration settings and different tasks were given in the DRLMOTS scheduler. The workloads are taken from by the google job datasets Tables 13, 14, 15. Table 13 represents the fault tolerance of small dataset for this we plot a result shown in Fig. 14.The proposed model DRLMOTS was evaluated against existing baseline algorithms: Convolution neural network (CNN), and long short-term memory (LSTM), gated graph convolution network (GGCN) DRLMOTS ran for 100 iterations. Generated fault tolerance 300, 600, 900, 1200, 1500 tasks for CNN algorithm are 98.4, 96.6, 92.9, 89.3, 87.5 respectively. Generated fault tolerance 300, 600, 900, 1200, 1500 tasks for LSTM algorithm 88.3, 84.9, 82.1, 80.2, 78.4 are respectively. Generated fault tolerance for 300, 600, 900, 1200, 1500 tasks for GGCN algorithm 80.2, 77.4, 74.7, 72.8, 70.2 are respectively. Generated fault tolerance for 300, 600, 900, 1200, 1500 tasks for DRLMOTS are 74.2, 72.4, 71.8, 69.6, 67.2 respectively. Table 14 represents the fault tolerance of medium dataset for this result shown in the Fig. 15. The proposed model DRLMOTS was evaluated against existing baseline algorithms: Convolution neural network (CNN), and long short-term memory (LSTM), gated graph convolution network (GGCN) DRLMOTS ran for 100 iterations. Generated fault tolerance 1800, 2100, 2400, 2700, 3000 tasks for CNN algorithm 82.1, 79.8, 77.4, 73.8, 68.4 are respectively. Generated fault tolerance 1800, 2100, 2400, 2700, 3000 tasks for LSTM algorithm 76.3, 71.5, 69.1, 66.4, 63.9 are respectively. Generated fault tolerance for 1800, 2100, 2400, 2700, 3000 tasks for GGCN algorithm 68.4, 66.2, 64.9, 61.1, 59.5 are respectively. Generated fault tolerance for 1800, 2100, 2400, 2700, 3000 tasks for DRLMOTS are 65.9, 63.6, 61.3, 59.4, and 56.8 respectively. Table 15 represents the fault tolerance of large dataset for this result was shown in the Fig. 16. The proposed model DRLMOTS was evaluated against existing baseline algorithms: Convolution neural network (CNN), and long short-term memory (LSTM), gated graph convolution network (GGCN) DRLMOTS ran for 100 iterations. Generated fault tolerance 3300, 3600, 3900, 4200, 4500 tasks for CNN algorithm are 65.2, 62.7, 59.4, 57.2, 52.9 respectively. Generated fault tolerance 3300, 3600, 3900, 4200, 4500tasks for LSTM algorithm are 59.5, 57.3, 55.7, 53.5, and 49.2 respectively. Generated fault tolerance for 3300, 3600, 3900, 4200, 4500 tasks for

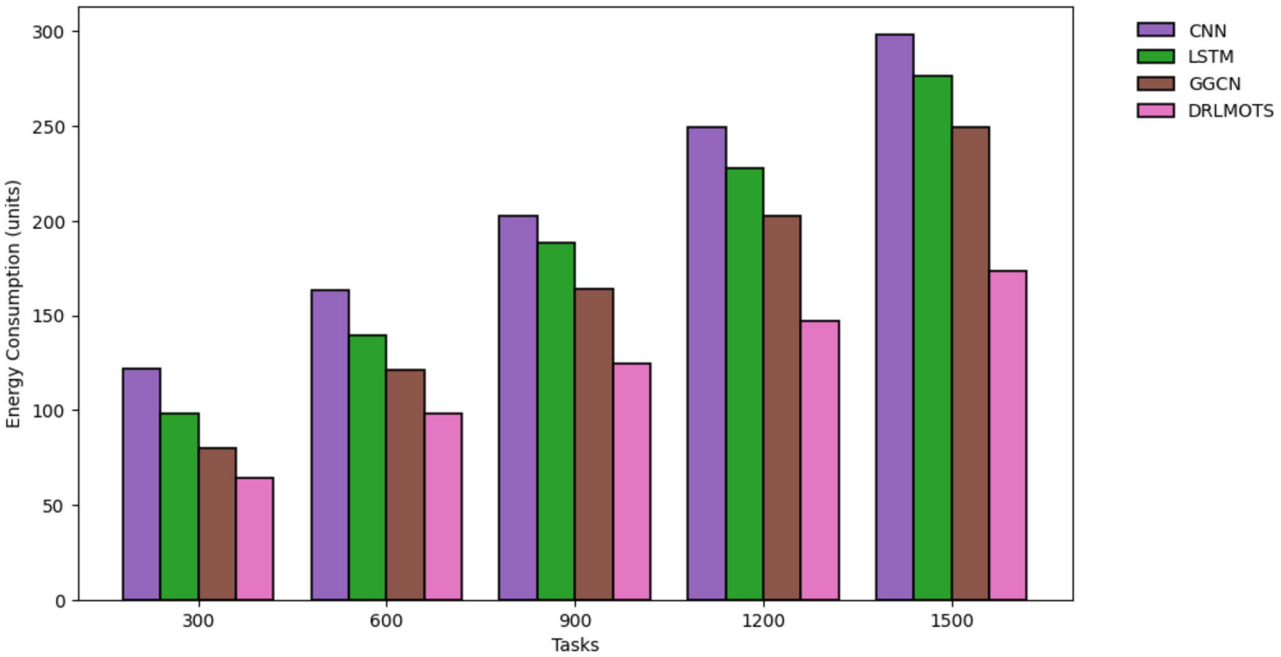

Fig. 11 Energy consumption of DRLMOTS for small dataset

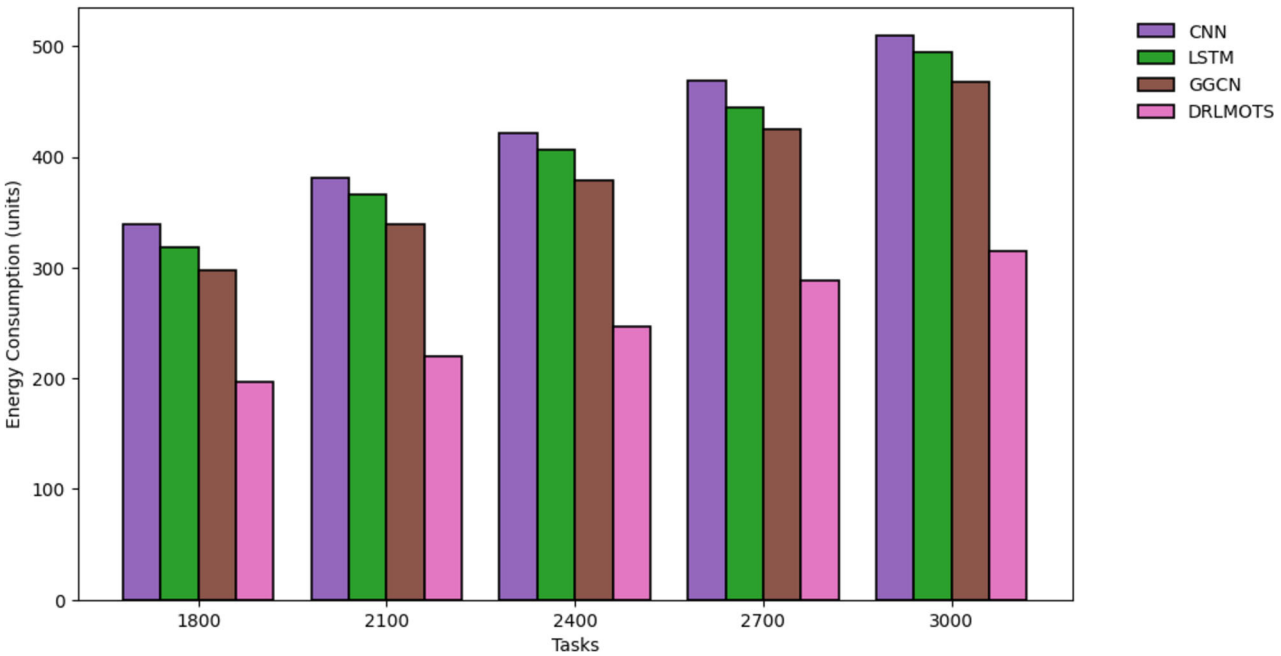

Fig. 12 Energy consumption of DRLMOTS for medium dataset

GGCN algorithm are 55.3, 52.6, 50.8, 48.2, and 45.1 respectively. Generated fault tolerance for 3300, 3600, 3900, 4200, 4500 tasks for DRLMOTS are 51.1, 49.3, 45.8, 43.6, and 40.3 respectively. The table below clearly illustrates that our suggested DRLMOTS model method tested over the CNN, LSTM, and GGCN algorithms and from simulation results of fault tolerance is reducing mention over the algorithms.

6.6 Result analysis and discussion

In result analysis and discussion section, we discuss about the consequence of boundaries and their further develops over the standard calculations CNN, LSTM, GGCN. We as of now notice about of our proposed calculation DRLMOTS model over the current calculations for boundaries energy consumption, makespan and fault

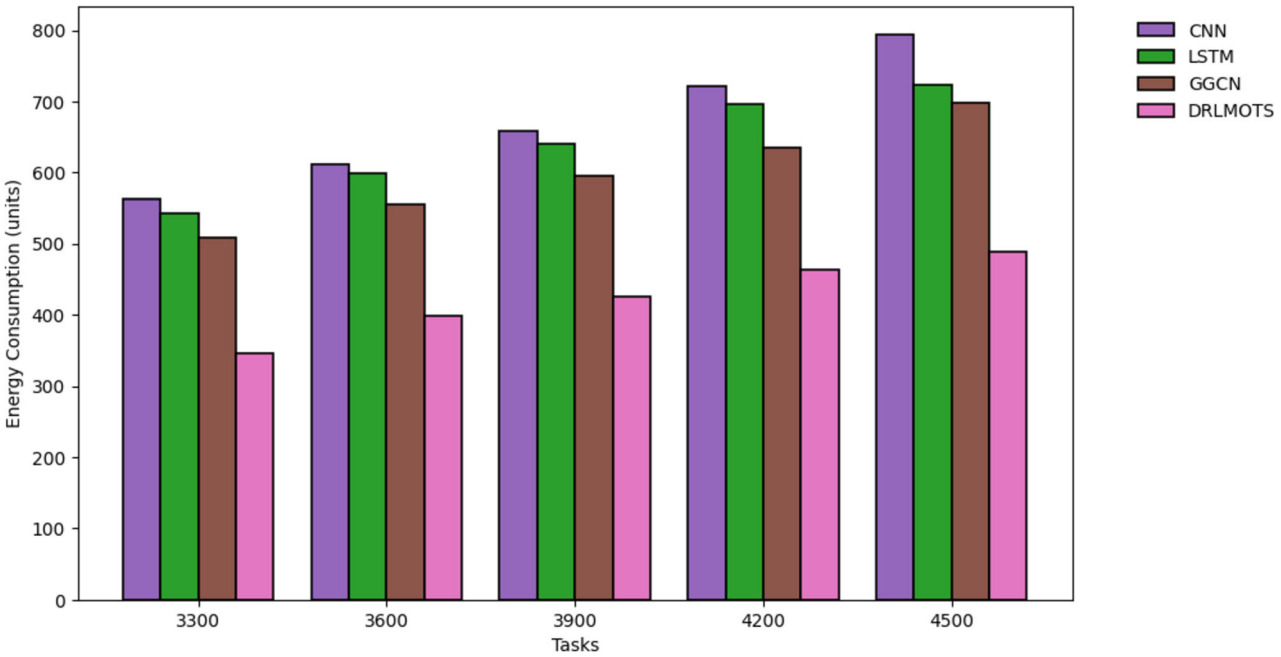

Fig. 13 Energy consumption of DRLMOTS for large dataset

Table 13 Evaluation of fault tolerance for small dataset

Tasks	CNN	LSTM	GGCN	DRLMOTS
300	98.4	88.3	80.2	74.2
600	96.6	84.9	77.4	72.4
900	92.9	82.1	74.7	71.8
1200	89.3	80.2	72.8	69.6
1500	87.5	78.4	70.2	67.2

Table 14 Evaluation of fault tolerance for medium dataset

Tasks	CNN	LSTM	GGCN	DRLMOTS
1800	82.1	76.3	68.4	65.9
2100	79.8	71.5	66.2	63.6
2400	77.4	69.1	64.9	61.3
2700	73.8	66.4	61.1	59.4
3000	68.4	63.9	59.5	56.8

Table 15 Evaluation of fault tolerance for large dataset

Tasks	CNN	LSTM	GGCN	DRLMOTS
3300	65.2	59.5	55.3	51.1
3600	62.7	57.3	52.6	49.3
3900	59.4	55.7	50.8	45.8
4200	57.2	53.5	48.2	43.6
4500	52.9	49.2	45.1	40.3

tolerance. The above Tables 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15 addresses the boundaries work on the makespan, energy consumption, and fault tolerance. Table 4, 5, 6 addresses our proposed DRLMOTS model reduce the makespan over contrasted existing gauge calculations and various responsibilities from google occupations of fixed VMs.Table 7, 8, 9 addresses our proposed DRLMOTS model reduce the makespan over contrasted existing benchmark calculations and various responsibilities from google jobs dataset of liner VMs.Table 10, 11, 12 addresses our proposed DRLMOTS model reduce the energy consumption over the existing algorithms and various responsibilities from google occupations. Table 13, 14, 15 addresses our proposed DRLMOTS model improves fault tolerance over contrasted existing calculations and various responsibilities from google jobs datasets (see Tables 16, 17, 18, 19).

Proposed DRLMOTS algorithm has advantages over existing mechanisms CNN, LSTM, GGN by considering task characteristics named length, processing capacities and addressing multiple objectives named makespan, fault tolerance, consumption of energy which are crucial in the scheduling process of cloud-fog paradigm. Moreover that, even we have increased tasks rapidly, our proposed scheduler DRLMOTS is addressed the above said metrics in an efficient manner over the other algorithms.

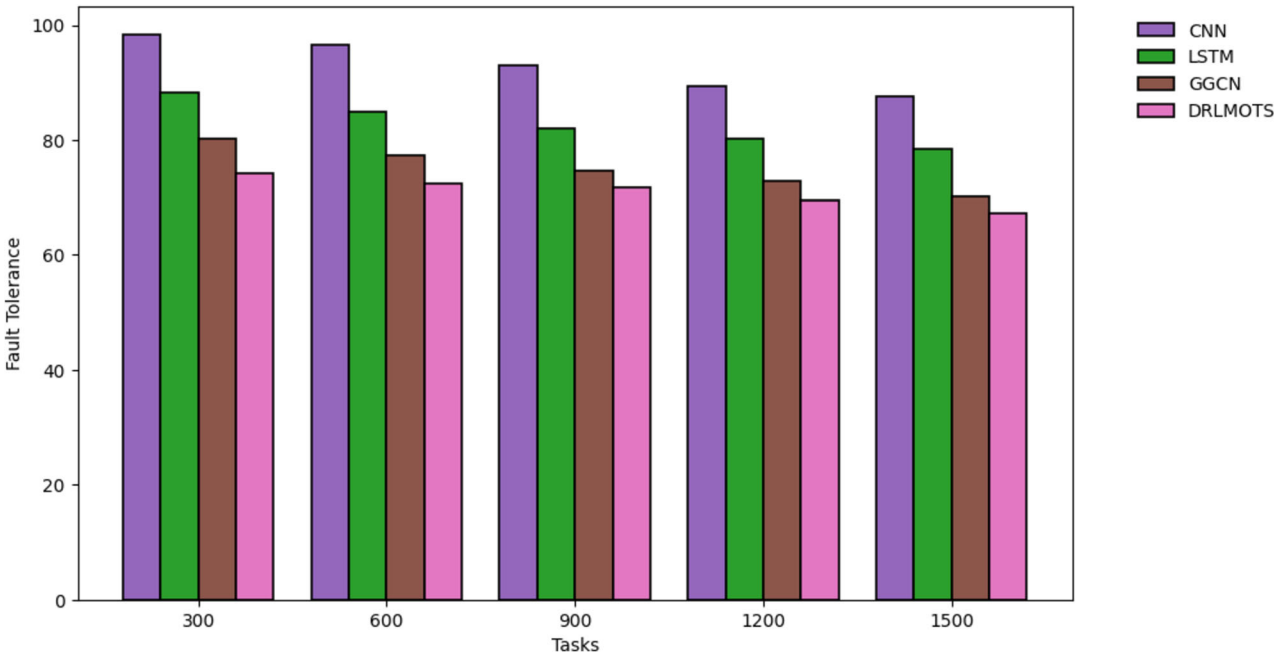

Fig. 14 Fault tolerance of DRLMOTS for small dataset

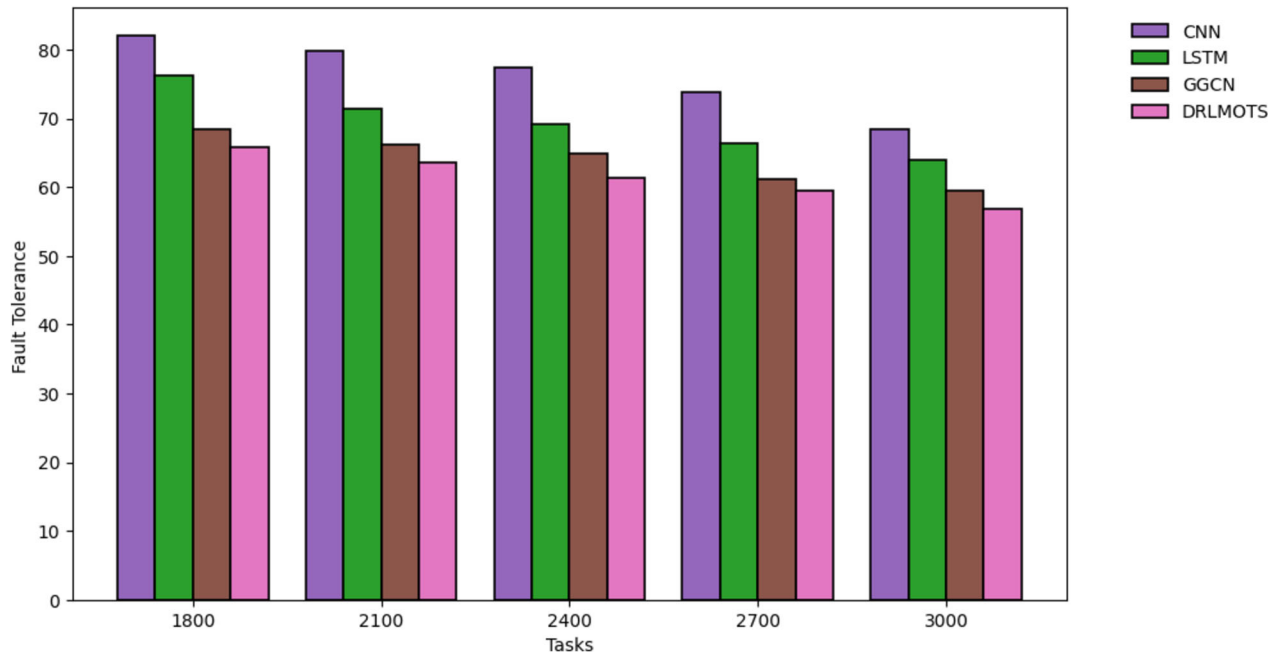

Fig. 15 Fault tolerance of DRLMOTS for medium dataset

7 Conclusion and future work

Task scheduling in cloud computing poses a significant challenge due to the diverse range of tasks with varying lengths and runtime capacities. Precisely allocating these tasks to suitable virtual resources is particularly difficult, especially when dealing with computationally intensive and time-sensitive tasks. Thus, we choose for Fog computing as an expansion to schedule heterogeneous jobs that are both time-sensitive and need significant computational resources, depending on the priorities determined at the task manager level. This study introduces a multi-objective task scheduler that prioritizes workloads and virtual machines based on energy efficiency and fault tolerance. The scheduler is coupled with a DQN model, which utilizes reinforcement learning techniques. The proposed

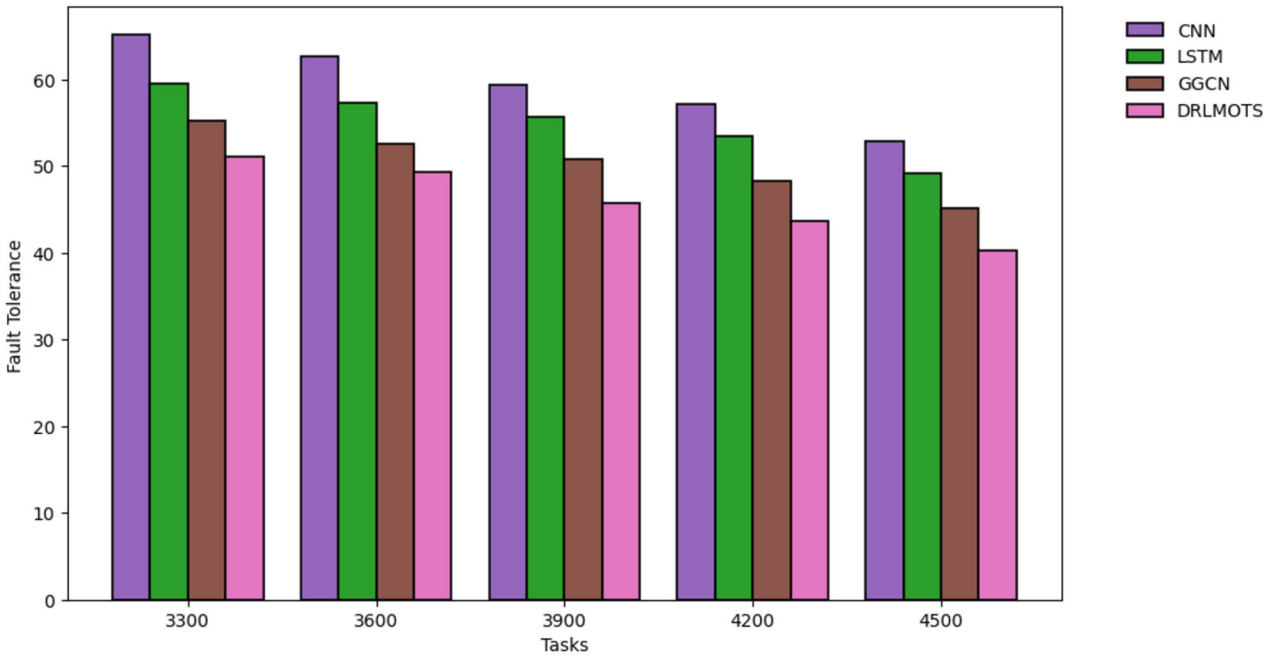

Fig. 16 Fault tolerance of DRLMOTS for large dataset

Table 16 Case1 Improvement of makespan for DRLMOTS model over existing algorithms

Tasks	CNN	LSTM	GGCN
300	42.12%	30.17%	21.50%
600	44.14%	28.89%	8.65%
900	44.90%	33.90%	19.55%
1200	40.31%	19.32%	9.64%
1500	33.34%	20.00%	10.55%
1800	29.48%	23.79%	8.57%
2100	26.24%	22.32%	5.24%
2400	24.46%	21.13%	6.63%
2700	14.90%	11.20%	7.39%
3000	17.11%	11.72%	5.24%
3300	15.78%	10.87%	4.02%
3600	14.07%	9.37%	7.65%
3900	17.85%	12.26%	2.69%
4200	18.61%	13.14%	3.96%
4500	18.75%	14.52%	4.50%

Table 17 Case2 Improvement of makespan for DRLMOTS model over existing algorithms

Tasks	CNN	LSTM	GGCN
300	42.44%	30.61%	21.58%
600	41.53%	29.19%	19.96%
900	42.73%	30.96%	21.97%
1200	43.84%	32.40%	24.11%
1500	44.78%	33.30%	25.61%
1800	44.10%	32.64%	24.58%
2100	43.43%	31.72%	23.53%
2400	43.32%	32.41%	24.61%
2700	42.88%	31.65%	23.92%
3000	44.02%	32.23%	25.68%
3300	44.06%	32.72%	25.00%
3600	43.84%	32.40%	25.36%
3900	44.25%	32.56%	24.42%
4200	44.14%	33.04%	25.50%
4500	43.98%	32.68%	25.44%

DRLMOTS algorithm builds schedules by considering the prioritization of activities and determines whether to schedule them on fog nodes or cloud nodes. The proposed Dynamic Resource Allocation and Load Monitoring System (DRLMOTS) is constructed using the SimPy framework. The workload provided to the scheduler is a real-time dataset called "Google Jobs," which contains a varying number of tasks. In order to conduct extended simulations, we partitioned the whole dataset into three categories: regular, medium, and big datasets. We then performed simulations by altering the number of virtual machines (VMs). We compared our proposed Deep Reinforcement Learning-based Multiple Object Tracking System (DRLMOTS) against baseline methods including Convolutional Neural Network (CNN), Long Short-Term Memory (LSTM), and Graph Convolutional Neural

Table 18 Improvement of energy computation for DRLMOTS model over existing algorithms

Tasks	CNN	LSTM	GGCN
300	47.22%	34.56%	19.36%
600	39.85%	29.63%	19.01%
900	38.54%	33.78%	23.82%
1200	41.03%	35.54%	27.46%
1500	41.89%	37.33%	30.42%
1800	41.95%	38.26%	33.97%
2100	42.45%	39.96%	35.35%
2400	41.52%	39.39%	34.98%
2700	38.59%	35.27%	32.28%
3000	38.35%	36.39%	32.83%
3300	38.41%	36.17%	31.81%
3600	34.88%	33.52%	28.28%
3900	35.34%	33.46%	28.46%
4200	35.70%	33.44%	27.01%
4500	38.39%	32.33%	29.97%

Table 19 Improvement of fault tolerance for DRLMOTS model over existing algorithms

Tasks	CNN	LSTM	GGCN
300	24.59%	15.97%	7.48%
600	25.05%	14.72%	6.46%
900	22.71%	12.55%	3.88%
1200	22.06%	13.22%	4.40%
1500	23.20%	14.29%	4.27%
1800	19.73%	13.63%	3.65%
2100	20.30%	11.05%	3.93%
2400	20.80%	11.29%	5.55%
2700	19.51%	10.54%	2.78%
3000	16.96%	11.11%	4.54%
3300	21.63%	14.12%	7.59%
3600	21.37%	13.96%	6.27%
3900	22.90%	17.77%	9.84%
4200	23.78%	18.50%	9.54%
4500	23.82%	18.09%	10.64%

Network (GGCN). The simulation results clearly demonstrate that our suggested technique significantly reduced the makespan, rate of failures, and energy usage. In the future, we will evaluate the effectiveness of the DRLMOTS scheduler by implementing it in an open stack environment to assess its performance.

Author contributions Prashanth Choppara written the manuscript, simulated results and Sudheer Mangalampalli reviewed the work carried out by prashanth and validated the results.

Funding Open access funding provided by Manipal Academy of Higher Education, Manipal.

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no competing interests.

References

1. Sandhu, A.K.: Big data with cloud computing: discussions and challenges. Big Data Mining Anal. **5**(1), 32–40 (2021)
2. Islam, M., Reza, S.: The rise of big data and cloud computing. Internet Things Cloud Comput. **7**(2), 45 (2019)
3. Tyagi, R., Gupta, S.K.: A survey on scheduling algorithms for parallel and distributed systems. In: Mishra, A., Basu, A., Tyagi, V. (eds.) Silicon photonics & high performance computing. Advances in intelligent systems and computing, vol. 718. Springer, Singapore (2018)
4. Alizadeh, M.R., et al.: Task scheduling approaches in fog computing: a systematic review. Int. J. Commun. Syst.Commun. Syst. **33**(16), e4583 (2020)
5. Zhang, P., et al.: A fault-tolerant model for performance optimization of a fog computing system. IEEE Internet Things J. **9**(3), 1725–1736 (2021)
6. Kaur, A., Auluck, N.: Real-time trust aware scheduling in fog-cloud systems. Concurr. Comput. Pract. Exp. **35**(10), e7680 (2023)
7. Iftikhar, S., et al.: HunterPlus: AI based energy-efficient task scheduling for cloud–fog computing environments. Internet Things **21**, 100667 (2023)
8. Movahedi, Z., Defude, B.: An efficient population-based multi-objective task scheduling approach in fog computing systems. J. Cloud Comput. **10**(1), 1–31 (2021)
9. Ghobaei-Arani, M., et al.: An efficient task scheduling approach using moth-flame optimization algorithm for cyber-physical system applications in fog computing. Trans. Emerg. Telecommun. Technol. **31**(2), e3770 (2020)
10 Yadav, A.M., Tripathi, K.N., Sharma, S.C.: A bi-objective task scheduling approach in fog computing using hybrid fireworks algorithm. J. Supercomput.Supercomput. **78**(3), 4236–4260 (2022)

11. Ghanavati, S., Abawajy, J., Izadi, D.: Automata-based dynamic fault tolerant task scheduling approach in fog computing. IEEE Trans. Emerg. Top. Comput.Emerg. Top. Comput. **10**(1), 488–499 (2020)
12 Elaziz, A., Mohamed, L.A., Attiya, I.: Advanced optimization technique for scheduling IoT tasks in cloud-fog computing environments. Futur. Gener. Comput. Syst.. Gener. Comput. Syst. **124**, 142–154 (2021)
13. Montazerolghaem, A., et al.: An optimal workflow scheduling method in cloud-fog computing using three-objective Harris-Hawks algorithm. *2022 12th International Conference on Computer and Knowledge Engineering (ICCKE)*. IEEE (2022).
14. Swarup, S., Shakshuki, E.M., Yasar, A.: Task scheduling in the cloud using deep reinforcement learning. Procedia Computer Science **184**, 42–51 (2021)
15 Razaq, M.M., et al.: Fragmented task scheduling for load-balanced fog computing based on Q-learning. Wirel. Commun. Mob. Comput.Comput. (2022). https://doi.org/10.1155/2022/4218696
16 Cheng, F., et al.: Cost-aware job scheduling for cloud instances using deep reinforcement learning. Clust. Comput.. Comput. (2022). https://doi.org/10.1007/s10586-021-03436-8
17 Tong, Z., et al.: QL-HEFT: a novel machine learning scheduling scheme based on a cloud computing environment. Neural Comput. Appl.Comput. Appl. **32**, 5553–5570 (2020)
18. Choppara, P., Mangalampalli, S.: Reliability and trust aware task scheduler for cloud-fog computing using advantage actor critic (A2C) algorithm. IEEE Access (2024). https://doi.org/10.1109/ACCESS.2024.3432642
19. Gazori, P., Rahbari, D., Nickray, M.: Saving time and cost on the scheduling of fog-based IoT applications using deep reinforcement learning approach. Futur. Gener. Comput. Syst.. Gener. Comput. Syst. **110**, 1098–1115 (2020)
20. Sharma, M., Garg, R.: An artificial neural network based approach for energy efficient task scheduling in cloud data centers. Sustain. Comput. Inf. Syst. **26**, 100373 (2020)
21 Siddesha, K., Jayaramaiah, G.V., Singh, C.: A novel deep reinforcement learning scheme for task scheduling in cloud computing. Clust. Comput.. Comput. **25**(6), 4171–4188 (2022)
22. Swarup, S., Shakshuki, E.M., Yasar, A.: Energy efficient task scheduling in fog environment using deep reinforcement learning approach. Procedia Comput. Sci. **191**, 65–75 (2021)
23 Jin, C., et al.: Reinforcement learning-based intelligent task scheduling for large-scale IoT systems. Wirel. Commun. Mob. Comput.. Commun. Mob. Comput. (2023). https://doi.org/10.1155/2023/3660882
24 Razaq, M.M., et al.: Fragmented task scheduling for load-balanced fog computing based on Q-learning. Wirel. Commun. Mob. Comput.. Commun. Mob. Comput. (2022). https://doi.org/10.1155/2022/4218696
25. Abdel-Basset, M., et al.: Multi-objective task scheduling approach for fog computing. IEEE Access **9**, 126988–127009 (2021)
26 Mangalampalli, S., Karri, G.R., Elngar, A.A.: An efficient trust-aware task scheduling algorithm in cloud computing using firefly optimization. Sensors **23**(3), 1384 (2023)
27 Mangalampalli, S., Karri, G.R., Kose, U.: Multi objective trust aware task scheduling algorithm in cloud computing using whale optimization. J. King Saud Univ. Comput. Inf. Sci. **35**(2), 791–809 (2023)
28. Sindhu, V., Prakash, M.: Energy-efficient task scheduling and resource allocation for improving the performance of a cloud–fog environment. Symmetry **14**(11), 2340 (2022)
29. Mohanapriya, N., et al.: Energy efficient workflow scheduling with virtual machine consolidation for green cloud computing. J. Intell. Fuzzy Syst. **34**(3), 1561–1572 (2018)
30. Zhang, X., et al.: Energy-aware virtual machine allocation for cloud with resource reservation. J. Syst. Softw.Softw. **147**, 147–161 (2019)
31. Karthiban, K., Raj, J.S.: An efficient green computing fair resource allocation in cloud computing using modified deep reinforcement learning algorithm. Soft. Comput.Comput. **24**(19), 14933–14942 (2020)
32. Rjoub, G., et al.: Deep and reinforcement learning for automated task scheduling in large-scale cloud computing systems. Concurr Comput Pract Exp **33**(23), e5919 (2021)
33. Zhou, G., Tian, W., Buyya, R.: Deep reinforcement learning-based methods for resource scheduling in cloud computing: A review and future directions. arXiv preprint arXiv:2105.04086
34. https://data.mendeley.com/datasets/b7bp6xhrcd/1

Prashanth Choppara received B.Tech. from Acharya Nagarjuna University, in 2018 and the M.Tech. degree from JNTUK University, in 2021. He is present pursing Ph.D. in School of Computer Science and Engineering (SCOPE) VIT – AP University, Andhra Pradesh, India. His area of interests in research are Cloud computing, Fog computing, Machine learning, deep Learning, Reinforcement Learning.

Dr. S Sudheer Mangalampalli working as an Associate Professor in department of Computer Science and Engineering in Manipal Institute of Technology, Bengaluru, Manipal Academy of Higher education known as MAHE (Institution of eminence, Deemed to be university). He is a Passionate researcher and his research interests includes scheduling in Cloud Computing, Edge Computing, Fog Computing. Towards his profile, he is having 42 Publications, which are indexed in Scopus and SCI databases. He authored three text books. He is a member of IEEE, institution of engineers. He is an active reviewer for various SCI indexed journals.